
基于 LangGraph 与 Web 地图 MCP 服务 的旅行规划智能体设计

2026 年 7 月 20 日

摘 要

随着自由行的兴起,旅行者在规划行程时需分散的信息中反复比对,并兼顾时间、预算与偏好等多重约束。大语言模型虽具备较强的自然语言理解能力,但直接用于旅行规划仍存在流程编排难、空间推理弱、实时数据与领域知识不足、缺乏长期记忆等局限。针对此,本文设计并实现了一个以 LangGraph 状态图为编排核心、借助模型上下文协议接入异构数据源的智能旅行规划智能体。

本文首先构建了五阶段状态图工作流,将规划流程划分为意图识别、约束抽取、并行数据获取、行程编排与审查、餐饮住宿匹配五个环节,并通过条件路由、并行扇出与局部重试实现流程的可控执行与失败隔离。其次,基于 MCP 协议对高德地图、12306、途牛等异构服务进行统一封装,设计了支持 SSE 与 STDIO 双模传输的客户端抽象,并引入重试、熔断与降级机制,以提升外部工具调用的稳定性和实时数据的可用性。为弥补大模型在知识与推理上的不足,本文将大模型与符号化算法相结合,并对知识与记忆进行分层处理:1)设计融合假设文档嵌入与交叉编码器重排序的双层 RAG 机制,结合结构化知识库与向量检索,以提升领域事实准确性;2)设计混合启发式编排算法,将景点的分日聚类与日内路径优化分别建模为带容量约束的均衡 K-Means 问题与时段分桶旅行商问题,借助传统算法保证空间合理性;3)构建长短期分离的记忆系统,在意图识别中隐式记录用户偏好,并在跨会话时自动注入,提供个性化服务。

本文将所提方法在测试集上与仅使用大语言模型的基线进行了对比,相关质量指标均有明显改善。消融实验进一步显示,检索增强生成、均衡聚类、时段分桶等模块对系统整体效果均有贡献。上述结果表明,本文所构建的复合智能体架构,有助于缓解大语言模型在垂直领域复杂任务中的固有不足,可为出行即服务向智能化、个性化演进提供一定的参考。

关键词: 旅行规划智能体; LangGraph; 模型上下文协议; 检索增强生成; 长期记忆

Abstract

With the deep penetration of the Internet and rising disposable income, China's tourism industry has expanded steadily, and independent travel has replaced standardized package tours as the dominant travel mode. However, travelers who arrange their own itineraries usually face the double challenges of scattered information and multi-objective constrained optimization. To address these problems, this paper designs and implements an intelligent travel planning agent orchestrated by LangGraph state graphs, with unified access to heterogeneous data sources via the Model Context Protocol (MCP). The main contributions are as follows: (1) The multi-stage travel planning process is modeled into a directed cyclic graph with intent branch condition routing, which can realize local retries without error propagation. (2) Different kinds of external services (such as Amap, Tuniu, 12306) are encapsulated by MCP through SSE and STDIO protocols, the fields are standardized to relieve context dilution, and a fault-tolerant mechanism is also established. (3) A double-layer retrieval-enhanced generation architecture that combines a structured knowledge base and vector retrieval is put forward, in which HyDE and cross-encoder reranking are utilized with the purpose of enhancing retrieval relevance and reducing factual errors. (4) A dual-memory system is designed to simultaneously keep short-term dialogue states and long-term user preferences, integrating preference extraction into intention recognition so as to be seamlessly injected into the planning phase. End-to-end tests are conducted in four scenarios: conventional urban planning, retrieval for cities not covered in the knowledge base, multi-round negotiation with missing constraints, and user preference accumulation. The results show an average response latency of approximately 80 seconds and an external service call success rate over 95%, which meets practical interaction requirements and verifies the feasibility and robustness of the proposed architecture.

KEY WORDS: Travel Planning Agent; LangGraph; Model Context Protocol; Retrieval-Augmented Generation; Long-term Memory

目 录

摘 要.....	II
Abstract.....	III
目 录.....	IV
第一章 绪论.....	1
1.1 研究背景和意义.....	1
1.2 研究现状.....	3
1.2.1 国内外研究现状.....	3
1.2.2 现有研究的不足.....	7
1.3 本文研究内容.....	8
第二章 关键技术基础与理论框架.....	10
2.1 AI 智能体核心理论与架构.....	10
2.2 LangGraph 工作流编排框架	11
2.3 MCP 工具调用协议	12
2.4 RAG 知识增强技术	13
2.5 本章小结.....	13
第三章 旅行规划智能体需求分析与总体设计.....	15
3.1 需求分析.....	15
3.2 系统总体架构.....	15
3.3 工作流总体设计.....	17
3.3.1 五阶段工作流总体设计.....	17
3.3.2 共享状态的分层组织.....	18
3.4 本章小结.....	20
第四章 旅行规划智能体核心模块设计与实现.....	21
4.1 MCP 协议集成与外部工具接入	21
4.1.1 MCP 协议与统一客户端抽象	21
4.1.2 基于 SSE 的高德地图客户端.....	22
4.1.3 基于 STDIO 的途牛与 12306 客户端.....	22
4.2 双层 RAG 知识增强机制	23
4.2.1 离线索引构建.....	23
4.2.2 在线检索与生成.....	24
4.3 行程编排算法设计.....	27
4.3.1 问题描述与算法框架.....	28
4.3.2 景点时长信息抽取与内容富化.....	30
4.3.3 基于行程时间矩阵的均衡 K-Means 聚类.....	30
4.3.4 时段分桶与桶内旅行商问题求解.....	33
4.4 记忆系统设计.....	35
4.4.1 记忆分层架构与统一访问入口.....	35
4.4.2 长短期记忆的存储实现.....	36
4.4.3 偏好的自动沉淀与上下文注入.....	36
4.5 前端设计与交互展示.....	36
4.5.1 前端技术栈与整体架构.....	37

4.5.2 主要界面设计.....	38
4.5.3 流式响应渲染.....	41
4.6 本章小结.....	42
第五章 旅行规划智能体系统测试.....	43
5.1 测试环境与系统配置.....	43
5.2 测试方法.....	44
5.3 端到端功能测试.....	44
5.4 对比测试.....	49
5.5 消融实验.....	51
5.6 本章小结.....	52
第六章 总结与展望.....	55
6.1 设计成果与主要结论.....	55
6.2 设计特色与创新.....	57
6.3 不足与展望.....	58
参考文献.....	59

第一章 绪论

1.1 研究背景和意义

文旅消费在国民经济中的占比持续提升，自由行已逐步取代标准化跟团游成为大众出行的主流模式。旅行规划属于典型的多约束复杂决策问题。用户通过自然语言表述出行需求，规划者需在预算、时间、地理可达性与景点覆盖度等多重约束下寻求可行解，同时结合票务、景点、住宿及餐饮等动态信息进行决策。长期以来，该过程的内在复杂性导致其高度依赖人工经验。尽管现有在线旅游服务平台提供了搜索与预订入口，但本质上仍是离散信息的聚合界面，难以根据用户的自然语言需求直接生成端到端的可执行方案；规划者仍需在多个异构平台之间反复切换、手动整合各方信息并自行求解约束冲突，决策成本居高不下。

近年来，以 Transformer 架构为基础的大语言模型（Large Language Model, LLM）在自然语言理解与生成任务上取得了突破性进展，为从用户的自然语言表述中解析出行意图提供了关键技术基础。在此基础上，融合工具调用与多步推理的智能体（Agent）技术范式逐步成熟，使大语言模型得以与外部数据源、服务接口协同完成端到端任务，为旅行规划这类多阶段复杂任务的自动化求解开辟了新的技术路径。然而，将通用大语言模型直接应用于真实旅行规划场景仍面临多项固有挑战，构成了本研究亟需解决的核心问题：

复杂工作流的可控编排能力缺失。旅行规划从用户意图解析与约束抽取开始，进入外部数据的并行获取与行程编排，最终由结果审查环节决定是否需要局部回退，各阶段之间存在严格的前后依赖；一旦某一环节出错，理想做法是就近重试而非全流程重启。传统的链式调用与单一 ReAct 循环缺乏显式状态机与条件路由能力，难以表达此类带分支、扇出与回环的复合控制流。

空间与组合推理能力薄弱使行程结构难以保证。旅行规划在景点分日与日内路径两个尺度上分别对应均衡聚类与小规模旅行商问题（Travelling Salesman Problem, TSP）求解，涉及精确数值比较、坐标距离计算与组合优化，是大语言模型最不擅长的任务类型之一。Xie 等^[1]在 TravelPlanner 基准上的实测显示，以 GPT-4 为代表的大模型在旅行规划任务上的完整方案通过率仅为 0.6%，揭示了纯模型推理在长程空间约束求解上的能力上限。

实时数据获取能力受限。大语言模型的知识源自一次性预训练语料，其知识边界由训练截止时点固化，而旅行决策所依赖的票务库存、酒店价格、景区开放状态与气象条件均处于分钟级或日级的动态变化之中，模型自身无法感知训练截止之后的事实，亦不能在生成过程中主动获取实时数据，由此导致输出方案与真实世界状态显著偏离。

领域知识深度不足引发内容幻觉。通用预训练语料中关于具体景点的合理游览时长、推荐游览时序、季节性注意事项与本地化经验提示等长尾知识覆盖稀疏，模型在生成时易将易混景点名称张冠李戴、对游览时长产生数倍偏差，或编造不存在的票种与优惠政策，直接削弱行程方案的可执行性。

长期记忆缺失阻断跨会话个性化。大语言模型在默认对话形态下不具备跨会话状态保持机制，每次新对话均以无状态方式启动，用户在多轮交互中暴露出的常用出发城市、偏好航司与酒店品牌、饮食禁忌等隐式偏好无法被自动沉淀与复用，规划者在每次会话中均需重复输入相同的背景信息，与个性化定制的初衷相悖。

上述挑战表明，仅凭通用大语言模型自身难以胜任真实场景下的旅行规划任务，需在其外围设计涵盖流程编排、工具接入、知识增强与个性化记忆的系统架构，以模块化方式弥补模型本身的能力边界。本文选用 **LangGraph** 状态图编排框架作为 workflow 调度与失败隔离的承载层；外部服务接入则统一基于模型上下文协议 (**Model Context Protocol, MCP**)，将高德地图、12306 等异构数据源封装为标准化工具。为缓解模型自身的领域知识盲区，系统引入检索增强生成 (**Retrieval-Augmented Generation, RAG**) 技术，对结构化必游知识与非结构化攻略文本进行协同召回；跨会话偏好则由独立的分层记忆机制持久化存储，并在会话起始时自动注入上下文。针对空间推理薄弱这一问题，本文进一步在编排层内嵌均衡聚类与 **TSP** 求解算法予以弥补。各模块协同工作，构成面向自然语言交互的端到端旅行规划智能体。

本研究的意义可从三个层面展开。在理论与方法层面，本文针对长程多约束规划任务这一大语言模型的薄弱场景，将状态图编排、标准化工具协议、双层检索增强与分层记忆机制整合于同一系统，验证了该类复合架构的可行性，可为后续大模型智能体在垂直领域的实现提供参考。在应用价值层面，本研究所构建的智能体能够将分散于多个在线旅游服务平台与攻略社区的异构信息整合为端到端可执行行程，有望降低自由行规划成本，为在线旅游服务从信息聚合向行程生成的演进提供实践依据。在交通运输工程学科层面，本研究是出行即服务 (**Mobility as a Service, MaaS**) 理念由城市通勤多模式联程，向跨城旅游多模式联程及目的地端服务编排延伸的一次实践探索，通过自然语言入

口与统一协议整合铁路、民航、城市交通及目的地资源，使规划过程从单一交通方式优化演进为全链条服务整合，研究成果可为智慧交通领域全链条出行服务的拓展与应用提供参考。

1.2 研究现状

1.2.1 国内外研究现状

基于大语言模型的智能体在复杂决策与规划任务中所展现出的应用潜力，使其成为智慧文旅与智能交通交叉领域的重要研究方向。旅行规划任务同时涉及自然语言意图解析、多维度约束条件优化与实时地理空间计算等多类异构子问题，对智能体在工作流编排、外部工具协同与领域知识增强等方面的能力提出了综合性要求。围绕这一任务，现有研究主要沿智能体编排框架演进、规划与推理方法、检索增强生成、工具调用协议标准化以及旅行推荐系统五个方向展开。

智能体通常指能够感知环境、自主决策并执行动作以达成既定目标的计算实体。Weng^[2] 将基于大语言模型的智能体抽象为以大语言模型为推理核心，通过规划、记忆与工具调用三大模块与外部环境交互的统一架构，如图 1-1 所示。该架构已成为后续多数智能体研究与工程实现的概念性基础。在上述概念框架的指导下，已有多项综述工作对智能体领域形成了系统梳理。Wang 等^[3]从整体视角回顾了自主智能体的研究进展；Xi 等^[4]提出了由控制端、感知端与行动端构成的通用智能代理理论框架。早期研究主要聚焦于提示工程，致力于提升单体智能体的复杂推理能力。Wei 等^[5]提出的思维链（Chain-of-Thought, CoT）方法揭示了分步推理对复杂问题求解的显著增益，成为后续所有推理增强技术的基础。Yao 等^[6]在此基础上提出 ReAct 框架，引入“思考—行动—观察”的闭环机制，通过智能体与外部环境的交互来获取实时反馈，有效缓解了纯思维链推理的幻觉问题。Shinn 等^[7]提出的 Reflexion 进一步引入语言化反思机制，使得智能体能够在情景记忆中沉淀错误经验，实现无需更新模型权重的自我修正。然而，上述方法均以单体智能体为核心，在任务分解与多角色协同方面能力有限，难以应对真实场景中流程复杂、环节多元的任务需求。

相关研究随之向多智能体协作框架延伸。Wu 等^[8]提出的 AutoGen 支持不同角色智能体的灵活组合与交互，成为应用较为广泛的多智能体编排框架。Hong 等^[9]提出的 MetaGPT 将软件工程中的标准操作流程编码进提示序列，在代码生成等任务中取得了较高的完成率。然而，上述智能体框架在架构形态上多采用线性链式结构或基于角色的

对话流组织方式，对长程任务中的显式状态管理、条件分支与失败回退等复合控制流缺乏原生支持，难以满足旅行规划这类多阶段任务对 workflow 可控性与可恢复性的需求。表 1-1 对主流智能体框架的关注点与设计特征进行了对比。

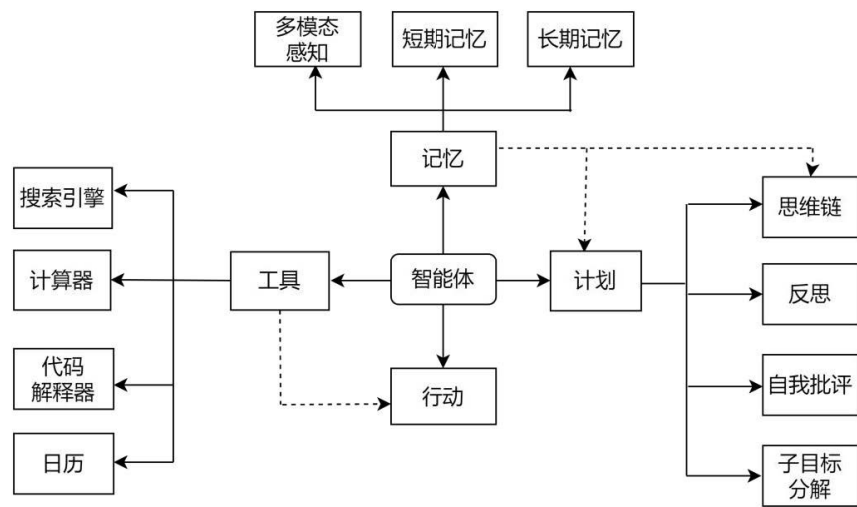


图1-1 大模型智能体框架

表1-1 主流智能体框架对比

框架	关注点	优势	适用场景
LangChain ^[10]	基于 LLM 的应用编排	工具链组合灵活，外部集成生态丰富	通用大语言模型应用开发
AutoGen ^[8]	多智能体对话	角色化交互，对话管理稳健	多智能体协作对话任务
CrewAI ^[11]	角色扮演与团队协作	模拟组织化分工模式	多角色组织化任务仿真
Semantic Kernel ^[12]	企业级应用集成	安全性与合规性支持完善	企业现有应用的智能化增强
LangGraph ^[13]	有状态多阶段 workflow	显式状态图建模，原生支持并行扇出、条件分支与局部重试，提供检查点持久化	长程多阶段、需细粒度控制流与容错的复杂任务

如表 1-1 所示，多数现有框架以链式调用或角色对话流作为主要组织形态，在工作流的可见性、可控性与可恢复性方面存在固有短板。LangChain 团队于 2024 年发布的 LangGraph^[13]在此基础上引入显式状态图与检查点机制，将智能体 workflow 建模为携带类型化共享状态的有向图，原生支持并行扇出、条件分支与局部重试等复合控制流结构，在细粒度 workflow 控制与容错执行方面提供了系统性的工程抽象，为生产级智能体系统的实现提供了支撑。

在智能体的规划与推理方面，早期工作主要借助提示工程引导大语言模型完成多步

规划。Wang 等^[14]提出的 Plan-and-Solve Prompting 在零样本设置下使模型按“先规划再求解”的模式工作，将复杂任务分解为多个子任务依次完成。Shen 等^[15]构建的 HuggingGPT 进一步提出任务规划、模型选择、执行、响应汇总的四阶段范式，使大语言模型能够整体协调调用多模态模型构成松耦合的工作流程，并且验证其作为通用任务调度器的可行性。Liu 等学者^[16]提出的 LLM+P 方案，将大语言模型作为 PDDL 问题的翻译器，使其与经典规划器相结合，以确保规划的逻辑完备性，从而弥补仅依靠模型进行推理时在严格约束求解方面存在的不足。在国内研究范畴内，秦龙等人^[17]构建中文复杂任务规划与执行的数据集 CTPaE 并提出 AutoPlan 框架，该框架依靠自主规划元任务序列且采用递进式 ReAct 逐步开展执行操作，在实验中对其所提算法的有效性进行了验证。尽管上述方法在数学推理以及通用任务规划基准方面已取得一定成效，但是当面临长程且多约束，高度依赖工具的复杂任务时，该方法的能力极限仍需进行评估。Xie 等人^[1]发布的 TravelPlanner 基准对这一状况进行了系统量化，结果显示 GPT-4 达成完整可行规划的成功率仅为 0.6%，这表明纯模型推理在工具调用失误，约束跟踪缺失以及长程任务偏移等多方面存在不足。此结果显示模型的内隐推理难以承担这类任务，需要在外部知识增强，工具调用以及工作流编排等许多层面上对模型能力进行系统性的补充。

在知识增强的维度层面，由于大语言模型存在垂直领域细粒度知识匮乏以及时效性不足的固有缺陷，检索增强生成（Retrieval Augmented Generation, RAG）已经成为知识增强的主流技术范式。Lewis 等人在文献^[18]中率先提出 RAG 框架，该框架通过融合参数化生成模型与非参数化外部知识库的方式，在无需更新模型权重的情况下，大幅提升了开放域问答的事实准确性，此举措成效明显。Gao 等人^[19]对 RAG 技术进行了全面综述，他们将 RAG 技术的演化进程细分为朴素 RAG，进阶 RAG 以及模块化 RAG 这三个阶段，并且明确指出查询改写，检索结果重排和上下文压缩是当前技术优化的关键方向所在。在生成端的反思与控制领域，Asai 等人^[20]在 20 年提出 Self-RAG 框架，其通过在生成进程中嵌入自反思标记来动态决策检索时机及引用方式，进而高效提高生成成果的事实一致性；Edge 等人^[21]提出的 GraphRAG 引入图结构组织检索片段，以此强化跨片段的推理能力。在嵌入与检索基础设施层面，Xiao 等人^[22]提出的 BGE 系列中文嵌入模型在 C-MTEB 这类中文检索基准上取得领先性能表现，并且为离线部署的领域 RAG 系统提供可行的本地化解决办法。但是当下众多 RAG 相关研究大多将目光聚焦于通用开放域的问答场景，对于旅游领域结构化城市知识和非结构化攻略文章并存的

双源知识形态，如何设计与之适配的检索召回、结果聚合以及多节点复用机制，当前尚无成熟技术方案可供依循。

在工具调用这一维度里，为了突破大语言模型在实时数据获取以及与外部服务交互方面的能力局限，工具调用与大模型的协同研究已经成为智能体领域的又一个关键方向。围绕智能体工具调用技术，当下的研究主要从以训练和检索为导向的方法论以及工具协议标准化这两个主要方面来推进。Schick 等人^[23]提出的 Toolformer，通过自监督的方式，使大语言模型在生成过程中掌握调用 API 的技能，进而推动智能体工具调用从人工设计模式向模型自主学习方向发展演进。在训练与检索相关的研究领域中，Qin 等人^[24]构建了 ToolBench 数据集并提出了 DFSDT 算法，基于此所训练出的 ToolLLaMA 模型极大地增强了工具调用方面的泛化能力。Patil 等人^[25]构建的 Gorilla 采用了检索增强微调的技术路径，使得大语言模型拥有调用主流深度学习平台海量 API 的能力。但是此类办法依赖众多标注数据与计算资源，且存在模型和工具的紧密耦合情况，难以适配工具频繁演变的真实生产场景。相比较而言工具协议的标准化为上述提及的问题提供了更具工程可实施性的解决办法。OpenAI 在 2023 年推出的 FunctionCalling 接口^[26]，构建了结构化函数调用的规范，极大程度简化了工具集成的开发流程。Anthropic 在 2024 年研制的 MCP^[27]，借助客户端—服务器的标准化通信架构，高效解决集成爆炸这一工程层面难题，且因其具备与具体模型分离的中立性及可扩展性，为通用工具层的抽象提供协议层支撑。

旅行推荐是典型的长程且受多种约束的开放域任务，其核心难题是在个性化体验与硬约束满足之间达成平衡，它也是检验智能体技术成熟度的关键应用场景。传统的旅行推荐方式是以协同过滤以及矩阵分解作为典型代表的。Koren 等人^[28]全面系统地阐述了矩阵分解推荐的相关方法，Cheng 等人^[29]则在基于位置的社交网络场景下提出了将地理与社交影响相融合的矩阵分解方法，此两者共同奠定了兴趣点（Point of Interest, POI）推荐的方法论起点。伴随深度学习的演进，Yang 等人^[30]和 Sun 等人^[31]相继将循环神经网络与长短期偏好建模引入，使得序列化推荐成为主流的发展趋势。在融合外部知识与混合建模的领域，国内研究者开展相关探索，贾中浩等人^[32]及刘智中等人^[33]分别将知识图谱与图神经网络引入个性化 POI 推荐中，且在稀疏交互数据情况下取得较理想效果。但是上述深度学习以及图神经网络的相关方法，主要针对的是单点推荐这一特定场景；对于真实旅行中复杂的多目标约束以及开放域行程生成这类需求，却难以有效应对。大语言模型演进为旅行规划开辟了全新的技术途径。Tang 等人在文献^[34]中提出的 ItiNera

将偏好感知的 POI 检索与空间聚类，TSP 求解这类优化算法相结合；其中用户意图理解以及行程文章生成为大语言模型所承担，空间布局和约束求解由优化算法负责，并对二者分层协同的工程可行性进行了验证；但是其仅聚焦于单城市单日场景，对多日跨城以及多模式交通的涵盖相对不足。Chen 等^[35]采用了 Gemini 结合优化算法的双阶段混合方案，以 LLM 解读定性目标，以算法保证旅行时间与营业时间等定量约束。在评测基准方面，Xie 等^[1]的 TravelPlanner 与 Wang 等^[36]的 TripTailor 是该领域当前较具代表性的两个评测基准。TripTailor 摒弃了既有基准依赖模拟数据的做法，基于超过 50 万个真实兴趣点与近 4000 条真实旅行路线构建评测集，并在约束满足之外引入可行性、合理性与个性化等多维评估指标；其实验结果显示，当前最先进大语言模型生成的行程中仅不足 10% 能达到人类水平，进一步暴露了纯模型方案在真实旅行规划中的能力局限。

1.2.2 现有研究的不足

综合上述五方面研究进展，现有工作在面向真实旅游场景的智能体系统构建方面仍存在五点核心局限。

(1) workflow编排与状态管理层面。LangChain 等主流框架主要采用链式调用的方式，对显式状态管理、条件分支与失败回退缺乏内置支持，难以表达多分支并行、阶段汇合与局部重试等复杂控制流。尽管 LangGraph 具备状态图建模的能力，但是在旅行规划场景中，对于各阶段依赖关系的描述方式、并发节点间共享状态的合并实现以及失败时的回退范围控制等问题目前尚无可借鉴的系统方案。

(2) 空间与组合推理层面。上述研究重点凸显出大语言模型在坐标距离计算，景点按日均衡聚类以及日内最优访问序列求解这类空间组合任务中存在系统性的不足。尽管目前已有研究尝试将 POI 检索与空间聚类，TSP 求解进行分层协同，且验证了大模型和求解器耦合的混合架构的可行性，但仅覆盖了单城市单日的场景，对于多日跨城，景点开放时段约束以及多模式交通衔接的拓展尚未充分开展。

(3) 异构工具集成调用层面。Toolformer、ToolLLaMA、Gorilla 等训练与检索导向方法，依赖大规模标注且模型与工具紧密绑定，因此难以适应接口频繁变化的真实场景。Function Calling、MCP 等标准化协议为工具与模型的分离提供了可行路径，但其在面向地图、票务、攻略等多来源旅游数据的领域中，落地实践数量仍较少，对 SSE 与 STUDIO 两类传输方式的统一封装、对原始 API 冗余字段的裁剪以及对第三方接口的容错处理，现有研究基础仍较薄弱。

(4) 领域知识注入层面。主流思路主要面向通用开放域问答，对旅游领域中结构化必游知识与非结构化攻略文本并存的情况，缺乏有针对性的协同检索机制；传统兴趣点推荐方法，如矩阵分解、图神经网络等所承载的 POI 数据，亦未与大模型的生成流程相互贯通。由此引发的景点张冠李戴，游览时间偏差以及虚构票种这类幻觉性质的问题，会直接削弱方案的可执行性。

(5) 跨会话个性化层面。现有的各项工作均将重点置于单次会话里的方案生成方面，对于跨会话的偏好提取，长期存储以及上下文自动注入几乎未进行探讨；若短期对话内容与长期用户画像采用同一种存储介质进行混用，临时语境极易对长期偏好造成污染，从而引发上下文污染的情况。用户每次会话时，仍需重复阐述出发城市，饮食禁忌及偏好航司等基础信息，已显现的偏好无法积累并实现复用。

1.3 本文研究内容

本文以“基于 LangGraph 与 Web 地图 MCP 服务的智能旅行规划智能体”为研究对象，融合大语言模型的自然语言理解能力与多源异构工具调用能力，实现端到端旅行行程的自动化生成。针对 1.2.2 节归纳的五点不足，本文确立五个核心研究问题并提出相应的设计方案。

(1) 复杂多阶段工作流的状态机建模。针对链式框架对状态管理与失败回退支持有限的问题，本文将旅行规划过程建模为有向回环状态图，依次包含意图识别、约束抽取、并行数据获取、行程编排与结果审查五个阶段，以类型化共享状态承载阶段间依赖，以条件路由表达分支与汇合，以单节点局部回环实现行程审查不通过时的就近重试。系统针对并发节点对共享状态的写入冲突，设计了自定义的状态合并规则，在保证并行加速的同时维持状态一致性，为长链路智能体提供可复用的编排参考。

(2) 空间分布与日内路径的混合启发式求解。针对大语言模型在空间组合推理上的系统性短板，本文将多日跨城行程编排拆分为景点分日聚类与日内访问序列求解两个子问题：前者基于行程时间矩阵设计带容量约束的均衡 K-Means 聚类，以兼顾景点地理分布不均与各日工作量需大致均衡的要求；后者将每日景点按时段分桶并在桶内求解小规模 TSP，同时把景点开放时段、餐饮节点与住宿位置作为软硬约束嵌入求解过程。创新点在于将传统组合优化算法以工具形式嵌入大模型工作流，形成大模型聚焦语义理解、传统算法承担空间求解、工作流统筹整体协同的分层架构，并将 ItiNera 单城市单日场景扩展至多日跨城与多模式交通衔接。

(3) 异构工具的统一协议封装与可插拔接入。针对训练导向工具学习方法与模型强耦合、维护成本高的问题，本文以 MCP 为统一接入标准，对高德地图、12306、途牛三类传输方式不同的外部服务进行协议层抽象，并在客户端层引入请求重试、熔断保护与字段裁剪等容错与压缩机制。本研究将 SSE 长连接与 STDIO 进程通信这两种传输方式收敛到同一抽象基类，并通过字段裁剪与归一化减轻原始 API 冗余对上下文窗口的占用，使工具层同时具备调用稳定性与上下文可控性，区别于既有 MCP 落地实践中对容错和字段处理较为简单的做法。

(4) 结构化知识库与向量检索的协同增强。针对通用 RAG 对旅游领域双源知识形态适配不足的问题，本文提出双层检索增强架构：以人工构建的城市结构化知识库承载高确定性的种子级 POI 与经典路线，以向量数据库承载长尾攻略文本。本系统设计种子景点桥接节点，以结构化知识库召回的 POI 为锚点驱动向量库进行语义补全，使两类知识在 POI 召回与最终响应生成两个阶段分别承担不同角色，相较 ItiNera 等仅依赖单一知识源的工作，能够同时覆盖知识库收录城市与长尾城市的细粒度内容。

(5) 跨会话用户偏好的沉淀与个性化复用。针对短期会话状态与长期偏好混用导致数据漂移的问题，本文采用长短期严格分离的分层记忆架构：短期会话状态由 LangGraph 检查点机制承载，长期偏好以用户为单位独立持久化存储，两类记忆在存储上完全分开。创新点在于将偏好提取任务合并入意图识别节点，借助同一次大模型推理同时完成意图分类与偏好抽取，避免额外推理调用带来的链路开销；提取出的偏好以系统消息形式自动注入后续会话上下文，在不增加用户额外操作的前提下完成跨会话偏好的积累。

围绕上述五个问题，本文设计并实现一个旅行规划智能体，以 LangGraph 为编排核心，通过 MCP 协议统一接入外部工具，引入混合启发式算法处理空间约束，借助双层 RAG 增强领域知识，并以分层记忆系统支撑个性化服务。系统能够接受自然语言输入并输出包含交通方案、逐日景点安排、餐饮推荐、住宿选择与预算核算的完整可执行行程，并在多轮交互中持续积累用户偏好。

第二章 关键技术基础与理论框架

本章梳理本研究依托的核心理论与技术基础。首先建立人工智能体的概念框架，进而依次介绍支撑系统实现的三项关键技术：LangGraph 工作流编排框架、MCP 工具调用协议与 RAG 知识增强技术，为后续系统设计与实现奠定理论基础。2.1 节阐述 AI 智能体的定义、核心理论与基本架构，明确以大语言模型为认知核心的智能体范式；2.2 节介绍 LangGraph 的状态图编排模型，构成系统编排层的技术支撑；2.3 节阐述 MCP 的设计架构与通信机制，确立外部工具标准化接入的规范基础；2.4 节论述 RAG 的核心流程与向量检索原理，为领域知识增强模块提供理论依据。

2.1 AI 智能体核心理论与架构

人工智能体（AI Agent）指能够感知环境、自主决策并采取行动以达成既定目标的计算实体。在经典人工智能理论中，智能体的运行被刻画为持续运行的“感知—决策—行动”循环：其从环境中获取观测信息，依据内部状态与目标进行推理，并通过执行器对环境施加影响。早期智能体多依赖规则系统或强化学习策略，其感知与决策能力受限于预设规则或特定任务的训练范围，难以应对开放域的复杂任务。

大语言模型的出现为智能体的构建提供了新的认知基础。借助在海量文本上的自监督预训练，大语言模型获得了通用的语言理解、知识表征与初步推理能力，并在参数规模增长至一定量级后呈现出指令跟随与少样本学习等涌现特性。以大语言模型作为认知核心，智能体得以将自然语言指令解析为内部目标，并以语言形式完成推理与规划。然而，单纯的语言模型仅能生成文本，缺乏与外部环境交互的能力。工具调用机制的引入使模型能够生成结构化的调用请求，并由运行时执行后将结果回填至上下文，从而形成“生成—执行—观察”的闭环，大语言模型由此从封闭的文本生成器转变为具备行动能力的智能体内核。

基于大语言模型的智能体通常由规划、记忆、工具与行动四类核心模块构成。规划模块负责任务分解与推理编排，其常见范式包括交替生成思考与行动的 ReAct 框架，以及将任务显式拆分为规划与执行两阶段的 Plan-and-Execute 范式，部分框架进一步引入反思机制以借助失败经验实现自我修正。记忆模块区分短期记忆与长期记忆，前者维护当前任务的上下文状态，后者沉淀跨会话的知识与用户偏好。工具模块以标准化接口封装外部函数与异构数据源，赋予智能体访问实时信息与外部状态的能力。行动模块则

将决策结果转化为对环境的实际操作，并将执行反馈传回认知核心，驱动循环的持续推进。

随着可调用工具与任务环节的不断增多，如何对智能体内部的推理、行动与状态流转进行有效编排，逐渐成为系统设计的关键问题。为此，需要引入具备分支控制、循环执行与状态持久化能力的编排框架对智能体的执行流程加以组织，相关技术原理详见 2.2 节。

2.2 LangGraph workflow编排框架

LangGraph 是 LangChain 团队于 2024 年开源的智能体 workflow编排框架。相较于 LangChain 表达式语言（LangChain Expression Language, LCEL）所构建的静态有向无环图，LangGraph 引入显式状态机抽象，将智能体 workflow建模为携带类型化共享状态的有向图，原生支持循环重试、条件分支与跨节点可变共享状态，可表达并行扇出（Fan-out）、扇入（Fan-in）以及嵌套子图等复杂编排形态。相比 AutoGen、CrewAI 等智能体编排框架，LangGraph 在长程多阶段流程的表达力与状态可控性方面更契合本研究的旅行规划场景。LangGraph 的核心抽象由状态（State）、节点（Node）与边（Edge）三部分构成。

状态是 LangGraph 的全局共享数据容器，在图的整个生命周期内被所有节点访问并逐步填充。StateGraph 实例化时需通过类型注解预先定义状态的数据结构，各节点的输入均从该状态对象读取，输出则以局部更新的形式合并回全局状态。与 LCEL 的单向数据流不同，状态更新通过归并器（Reducer）实现增量合并而非整体覆盖，为并行节点对共享字段的安全写入提供了基础保障。节点是图的基本计算单元，每个节点封装独立的计算逻辑，对应一个接收状态对象并返回状态局部更新的同步或异步函数，使各节点天然解耦，可独立测试与跨图复用。边定义节点间的连接关系与执行流向：普通边建立顺序执行路径，条件边通过路由函数根据当前状态动态决定后续节点，返回单一节点名时实现条件分支，返回节点名列表时等价于动态并行扇出。此外，LangGraph 提供的检查点（Checkpoint）机制可对每一步执行后的状态进行持久化，支持进程中断后的恢复与跨会话的状态续接。本研究将旅行规划流程建模为含并行扇出与局部重试回环的状态图，依托 LangGraph 上述特性表达多阶段依赖与失败隔离逻辑，具体节点设计将在第四章详细阐述。

2.3 MCP 工具调用协议

模型上下文协议（Model Context Protocol, MCP）是 Anthropic 于 2024 年提出的开放协议，旨在统一大语言模型与外部工具、数据源之间的交互规范。在 MCP 提出之前，主流做法是基于各模型厂商专属的 Function Calling 接口直接声明工具，工具定义与宿主应用代码深度耦合，工具的增删或参数调整需同步修改应用层代码，且不同厂商接口格式不统一，导致工具代码难以跨平台复用。MCP 通过引入独立协议层将工具的定义、发现与调用逻辑从宿主应用中解耦，使工具服务可独立开发、部署与跨模型复用。

MCP 采用客户端—服务器架构。服务器端声明并实现工具集合，对外暴露工具名称、参数模式与功能描述；客户端运行于宿主应用内部，负责工具发现、调用请求构造与返回结果解析。服务器与宿主应用在进程或网络层面物理隔离，工具服务可独立迭代。在通信层面，MCP 针对不同部署场景定义了多种传输方式：本地工具采用标准输入输出（STDIO）传输，宿主进程以子进程形式启动服务器，通过标准流实现双向通信，适合将现有命令行程序快速包装为 MCP 服务器；远程工具采用基于 HTTP 的服务器发送事件（Server-Sent Events, SSE）传输，客户端通过 HTTP POST 请求触发调用，服务器以事件流形式返回结果，支持流式输出。在能力发现层面，客户端建立连接后通过 tools/list 接口查询服务器端工具的名称、描述与参数 JSON Schema，服务器新增或修改工具时，客户端可在下次连接初始化时自动同步；调用阶段则通过 tools/call 请求传递工具名称与实参，服务器按 MCP 规范返回结构化执行结果。MCP 与 Function Calling 的核心特性对比如表 2-1 所示。

综上，MCP 通过协议层解耦实现了工具定义、发现与调用的标准化，在多异构数据源接入且依赖频繁迭代的场景下，相较 Function Calling 具有更优的可维护性与可扩展性，因此被本研究选为统一接入高德地图、途牛业务接口与 12306 票务三类异构数据源的协议基础。

表2-1 MCP 与 Function Calling 主要特性对比

比较维度	Function Calling	MCP
工具定义位置	嵌入宿主应用代码	独立于 MCP 服务器
工具发现方式	静态声明，随请求传输	运行时动态查询
部署耦合度	工具变更需重新部署应用	工具变更不影响宿主应用
跨平台复用性	依赖特定模型接口格式	协议统一，与模型无关
token 开销	每次请求携带全量工具描述	工具描述由服务器维护，按需查询
传输方式	无独立传输层	STDIO / SSE / 流式 HTTP
异构数据源集成	需各自编写适配代码	统一接口封装，屏蔽实现差异

2.4 RAG 知识增强技术

大语言模型的知识边界受限于训练语料的截止时间与覆盖范围，在垂直领域最新信息、长尾事实及强地域性内容上易出现知识遗漏与幻觉生成。RAG 技术由 Lewis 等于 2020 年首次提出，其核心思想是在生成阶段显式注入与查询语义相关的外部文档片段，无需更新模型权重即可显著提升生成内容的事实准确性与时效性。标准 RAG 流程包含查询处理、语义检索、上下文增强与答案生成四个核心阶段，用户原始问题或经改写的检索式由嵌入模型编码为稠密向量后输入向量数据库，基于向量相似度返回 Top-K 个最相关的文档片段，将检索结果与原始问题拼接为结构化提示词，输入大语言模型生成最终答复。

文本嵌入模型与向量数据库是 RAG 系统的两类核心基础组件。文本嵌入模型负责将变长自然语言文本映射为定长稠密向量，使语义相近的文本在高维向量空间中具有较小的余弦距离，为语义相似度检索提供数学基础；本研究选用北京智源人工智能研究院开源的 bge-small-zh-v1.5 中文嵌入模型，该模型在中文海量文本嵌入基准（Chinese Massive Text Embedding Benchmark, C-MTEB）上具有优异的检索性能，参数量约 24M，输出向量维度为 512，单 CPU 即可本地部署，无需依赖云端 API。向量数据库则负责高维稠密向量的存储与近似最近邻检索；本研究采用 Chroma 团队开源的轻量级嵌入式向量数据库 ChromaDB，其底层基于层次化导航小世界（Hierarchical Navigable Small World, HNSW）索引结构，在召回精度与查询时延之间取得良好平衡，同时支持元数据过滤与本地嵌入式部署。

本研究在通用 RAG 范式之上构建结构化城市知识库与本地向量检索协同的双层架构，针对旅游领域结构化必游知识与非结构化攻略文本并存的双源特征进行差异化复用，具体设计将在 4.2 节详细阐述。

2.5 本章小结

本章系统梳理了支撑本研究的核心理论与技术基础。首先，界定了 AI 智能体的概念内涵，阐述了以大语言模型为认知核心、由规划、记忆、工具与行动四类模块构成的智能体基本架构，明确了“生成—执行—观察”闭环这一核心运行机制，为理解后续各项技术在整体系统中的定位提供了理论框架。其次，阐述了 LangGraph 工作流编排框架的核心抽象，包括其以类型化共享状态（State）为容器、节点（Node）为计算单元、边（Edge）定义流向的模型，并重点说明了其支持条件分支、并行扇出与局部重试的特

性，这为后续将复杂旅行规划流程建模为可控状态图奠定了基础。随后，介绍了模型上下文协议（MCP）的客户端—服务器架构及其通过标准化协议解耦工具与模型的优势，对比了其与 Function Calling 在可维护性、跨平台复用性方面的差异，明确了选用 MCP 作为异构数据源统一接入协议的理由。最后，概述了 RAG 技术的基本流程与核心组件，为第四章设计针对旅游领域的双层知识增强机制提供了理论依据。本章内容为后续系统的架构设计、模块实现与集成提供了清晰的技术选型支撑。

第三章 旅行规划智能体需求分析与总体设计

本章在前一章相关技术综述的基础上，从需求分析出发，系统阐述旅行规划智能体系统的总体设计。3.1 节梳理系统的功能性与非功能性需求；3.2 节提出分层架构与技术选型对照；3.3 节描述五阶段工作流的总体形态及其共享状态的分层组织方式。

3.1 需求分析

该系统面向自然语言交互的旅行规划场景，外部参与者主要包括终端用户与外部 MCP 工具服务方两类。终端用户通过 Web 前端发起请求，外部 MCP 工具方包括高德地图、途牛业务接口与 12306 票务服务，系统根据业务需求以异步方式调用其能力。

基于上述参与者，系统功能性需求可归纳为一条核心交互闭环与四类辅助交互意图。核心交互闭环为端到端行程生成：用户以自然语言描述出行需求，系统在单一会话内依次完成意图识别、约束抽取与校验、多源数据获取、行程编排、餐饮住宿匹配、预算核算及最终响应渲染，输出包含逐日景点安排、交通方案、餐饮推荐、住宿建议与预算明细的完整旅行方案；当用户输入信息缺失或存在冲突时，系统应主动发起追问，待信息补全后继续执行规划流程。四类辅助意图分别为偏好更新、记忆查询、信息查询与行程调整，依次覆盖长期偏好的显隐式更新、跨会话历史回溯、无需完整规划的轻量问答及对已有行程参数的增量修改。上述四类辅助意图与核心闭环共享统一的意图识别入口，由意图分流逻辑分发至对应处理路径。

非功能性需求聚焦于系统运行效率与质量，主要涵盖响应延迟、可用性、可扩展性与数据持久化四个维度。响应延迟方面，端到端单次行程生成需在用户可接受的时间范围内完成，并通过流式输出技术降低用户主观等待感；可用性方面，当单一外部数据源出现限流、超时或服务异常时，系统应实现局部功能降级而非整体服务中断，保障其他数据通道与下游节点正常运行；可扩展性方面，外部数据源与生成式能力均应采用可插件式设计，新增地图、票务或知识源时不应导致现有节点与状态结构的破坏性变更；数据持久化方面，会话级上下文需支持进程或服务重启后的恢复能力，长期用户偏好与历史行程需以独立于会话的方式持久存储，二者共同支撑跨会话的连续交互体验。

3.2 系统总体架构

为同时满足自然语言交互、多智能体协同与异构数据源接入的核心需求，本系统采用五层分层架构设计，自顶向下依次为交互层、编排层、智能体层、工具层与数据层，

整体架构如图 3-1 所示。

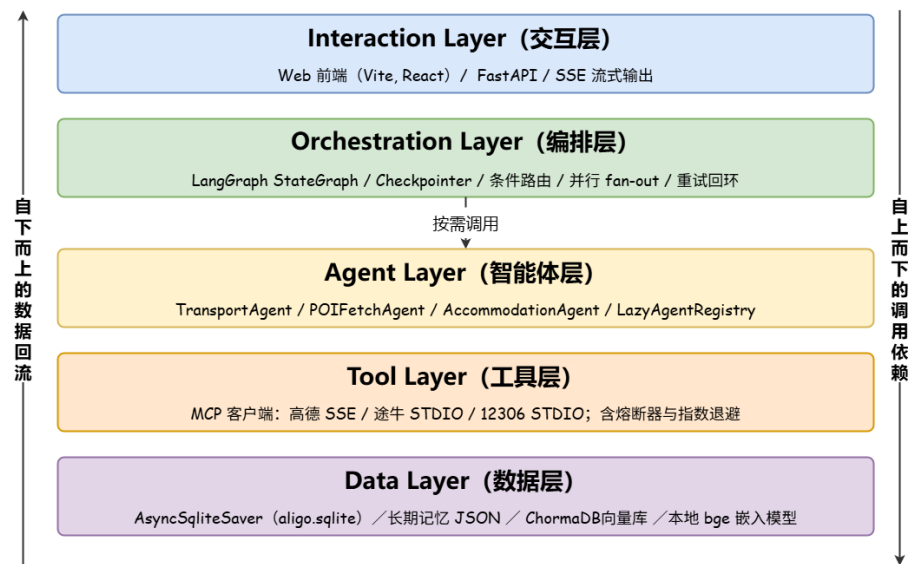


图3-1 系统总体架构图

交互层负责接收用户输入并返回系统响应，主要实现基于 FastAPI 服务器与 SSE 技术的 Web 接口。该层仅承担消息收发与流式渲染职责，不参与任何业务逻辑处理，从而实现前后端的解耦与独立演化。编排层以 LangGraph 状态图为核心载体，负责节点装配、条件路由、并行扇出与重试回环等控制流逻辑，并通过 Checkpointer 机制实现会话级共享状态的持久化。智能体层包含交通规划智能体、POI 检索智能体与住宿推荐智能体等领域专用智能体单元，这些智能体不直接出现在状态图拓扑中，而是由编排层节点根据业务需求动态调用。工具层基于 MCP 协议封装高德地图、途牛业务接口与 12306 票务服务三类异构外部资源，统一对外暴露异步调用接口，并通过熔断器与指数退避机制实现服务失败隔离与功能降级。数据层包括基于 SQLite 的会话检查点存储、基于 JSON 文件的长期用户偏好与历史行程存储，以及基于 ChromaDB 的非结构化旅游攻略片段向量存储，三者共同构成系统的持久化支撑体系。各层技术选型均经过多方案对比论证，最终决策如表 3-1 所示：

表3-1 技术栈选型对照表

层次	组件	最终选型
编排层	工作流引擎	LangGraph
模型层	大语言模型	doubao-seed-2-0-lite
工具层	工具协议	MCP
检索层	向量数据库	ChromaDB
嵌入层	中文嵌入模型	bge-small-zh-v1.5

持久化层	会话检查点	AsyncSqliteSaver
服务层	Web 后端框架	FastAPI
前端层	构建工具与框架	Vite, React

3.3 workflow 总体设计

3.3.1 五阶段 workflow 总体设计

本系统 workflow 按节点职责划分为五个主阶段与一个辅助阶段，依次为意图与约束抽取（P1）、并行数据获取（P2）、行程编排（P3）、行程审查与重试（P3.6）、餐饮住宿匹配（P4）及响应渲染（P5），整体架构以线性执行骨架为基础，融合并行扇出的延迟优化机制、局部重试的容错保障机制与预算驱动的约束自适应机制，形成多维度协同的复合 workflow 形态，如图 3-2 所示。

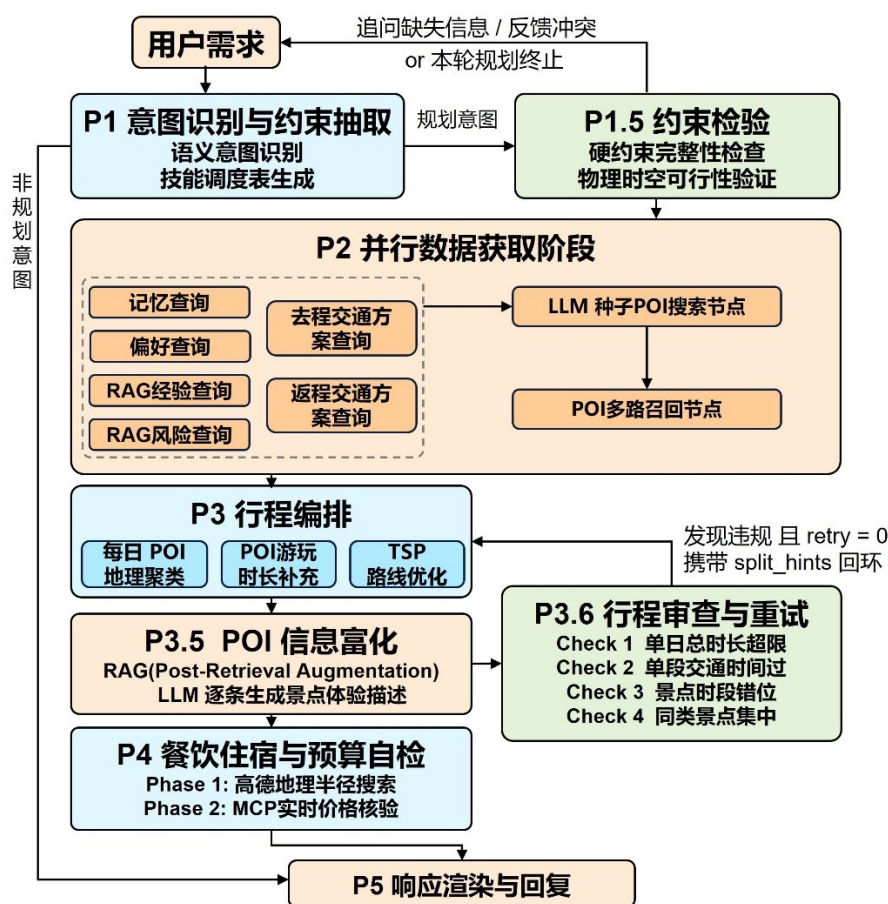


图3-2 workflow 架构

P1 为意图与约束抽取阶段，由意图识别、约束抽取、约束校验与协商四个节点串行构成。意图识别节点通过单次大语言模型（LLM）调用同步输出意图类型、关键实体与用户隐含偏好，并将偏好信息持久化至长期记忆。阶段结束后，条件路由函数依据意图类型实现 workflow 分流，使同一架构可同时支持完整行程规划与轻量信息查询两类核心

场景。

P2 并行数据获取阶段采用三路并行扇出结构，同时启动检索增强生成（RAG）检索、去程交通查询与返程交通查询任务。三路结果汇聚后，LLM 种子节点从 RAG 检索片段与城市知识库中提取兴趣点（POI）种子，再由 POI 搜索节点完成知识库感知的多路 POI 召回。该并行架构将数据获取端到端延迟由串行累加压缩为单路最大延迟，有效提升了系统响应速度。

P3 为行程编排阶段，行程规划节点对每日 POI 进行地理聚类，将单日 POI 访问顺序建模为旅行商问题（Traveling Salesman Problem, TSP）求解最短遍历路径，并估算各 POI 合理游览时长；P3.5 POI 信息富化节点并行获取 POI 描述与配图。P3.6 行程审查与重试阶段对行程草案执行规则校验：若检测到关键违例且未达重试上限，控制流回退至行程规划节点重新计算，回退前清空上一轮工具调用结果以避免状态污染；重试次数采用常量硬约束，防止编排与审查节点间形成无限循环。该局部重试机制将失败影响限定于单阶段，避免了全流程重启的计算开销。

P4 阶段负责餐饮住宿匹配与预算自检。餐饮节点以每日 POI 几何中心为基准查询周边餐厅，且独立于重试回环之外，避免重复调用地图 API 消耗配额；住宿节点生成每日高、中、低三档备选方案，按当日地理重心、抵达枢纽和目的地中心的优先级逐级扩大搜索范围，提升住宿匹配成功率。预算自检节点基于交通与住宿费用占总预算的比例进行判定。若占比超阈值且住宿档位可下调，则回退至住宿节点降档后重新校验；若已达最低档位仍超预算，则在最终响应中添加预算超限提示；若未超阈值则直接进入 P5 阶段。该分支实现了约束驱动的住宿档位自适应调整。P5 响应渲染阶段由单一节点完成最终响应组装，并将行程摘要持久化至长期记忆，为后续会话的偏好复用与个性化推荐提供数据支撑。

综上，本工作流程通过线性主路径保证执行可预测性，通过 P2 阶段并行扇出降低端到端延迟，通过 P3.5 阶段局部重试提升系统鲁棒性，通过 P4 阶段预算分支实现约束自适应调整，四类结构特征协同支撑了自然语言交互下可执行行程的生成能力。

3.3.2 共享状态的分层组织

在 LangGraph 范式下，状态图节点间不通过显式函数参数传递数据，而是共享全局状态对象。各节点从该对象读取输入字段，并以归并语义将计算结果回写至同一对象。本系统定义共享状态为 TravelGraphState，采用 TypedDict 与 Annotated 归并器结合的方式实现类型化描述，按数据职责划分为约束类、规划类、富化类与编排类四层结构，

如图 3-3 所示。

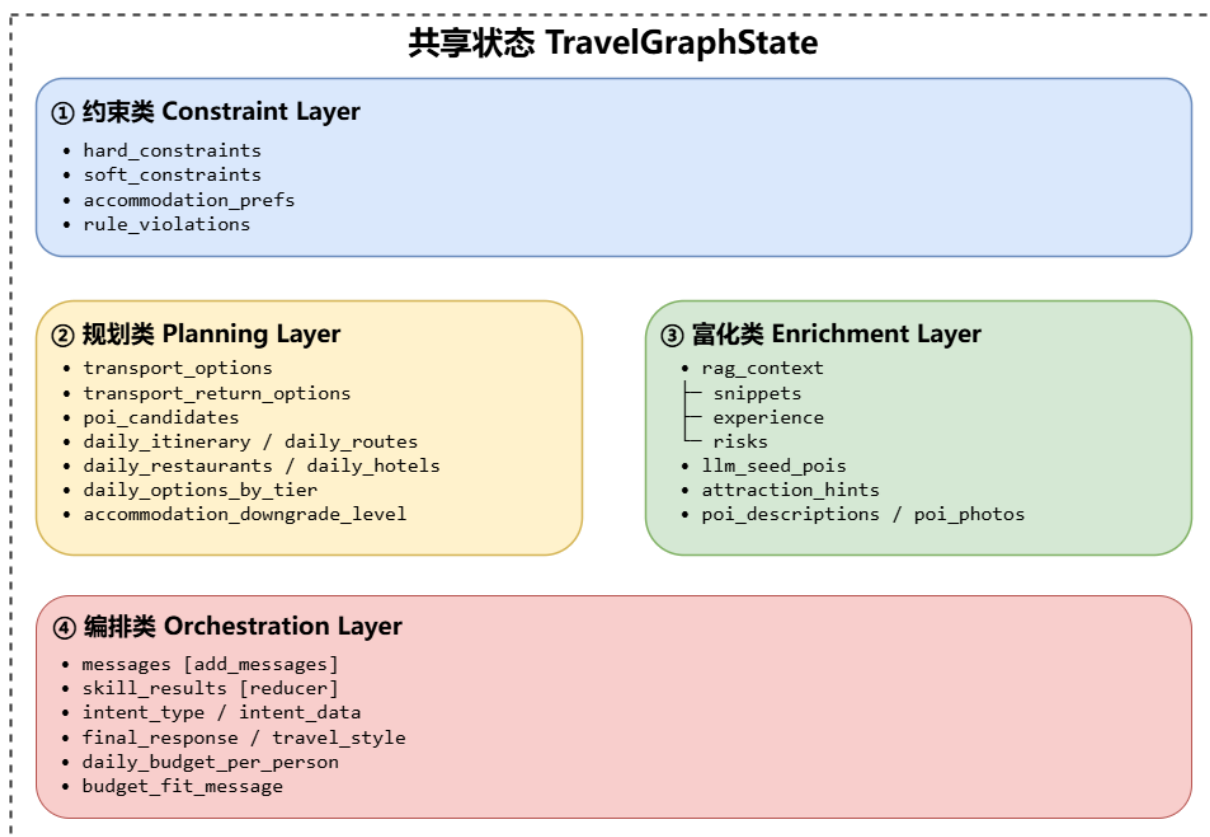


图3-3 TravelGraphState 共享状态分层结构

约束类承接用户需求边界，由 P1 阶段的约束抽取与约束校验节点写入，包含硬约束、软约束、住宿偏好以及规则违例集合四组字段。该类数据写入完成后对后续所有节点只读，构成行程生成的不变量约束基底，同时为后续节点的并发执行提供线程安全保障。

规划类存储行程生成过程中的中间数据，在 P2 至 P4 阶段逐步填充，主要字段包括往返交通候选、POI 候选池、逐日行程草案、逐日路线、逐日餐厅、逐日住宿三档备选及当前住宿降档次数。该类数据按上游至下游的顺序单调追加，下游节点仅依赖上游已写入字段，无循环依赖关系。

富化类存储知识增强结果，包括检索上下文容器、LLM 提取的 POI 种子集合、POI 搜索关键词提示及 POI 描述与配图。该类与规划类在生成阶段并行写入，响应渲染阶段可独立读取，互不干扰。

编排类服务于节点间路由判定与最终输出，包括 LangGraph 标准消息列表、意图类型与槽位、各模块工具调用结果、最终响应文本、旅行风格标识及预算提示文案。其中消息列表采用 LangGraph 内置的追加归并器，支持并发节点安全写入；工具调用结

果采用自定义归并器，支持重试场景下整体清空历史结果；其余字段默认采用覆盖写语义。差异化的归并语义配置为 3.3.1 节所述的并行扇出与局部重试机制提供了状态层面的实现基础，具体细节将在第四章节点级设计中阐述。

3.4 本章小结

本章从需求出发，完成了旅行规划智能体系统的总体设计。首先，进行了系统的需求分析，明确了以自然语言生成端到端可执行行程为核心的功能性需求，以及涵盖响应延迟、可用性、可扩展性与数据持久化的非功能性需求。在此基础上，提出了系统的总体架构，采用交互层、编排层、智能体层、工具层与数据层的五层分层设计，并给出了各层具体的技术选型对照表，确立了以 LangGraph 为编排核心、MCP 为工具协议、RAG 为知识增强的整体技术路线。继而，详细阐述了工作流的总体设计，将规划流程分解为意图与约束抽取（P1）、并行数据获取（P2）、行程编排（P3）与审查重试（P3.5）、餐饮住宿匹配（P4）、响应渲染（P5）五个主阶段，说明了其间的线性、并行与回环控制流。同时，定义了承载全局数据的分层共享状态（TravelGraphState），并说明了其按约束、规划、富化、编排分类的组织方式与归并语义。本章从抽象到具体，为后续各核心模块的详细设计与实现勾勒了完整的系统蓝图。

第四章 旅行规划智能体核心模块设计与实现

本章基于第三章提出的系统总体架构与 workflows 形态，针对 1.3 节归纳的核心问题，展开各核心模块的详细设计与工程实现。本章涉及的五模块在 workflows 中并非彼此孤立，而是通过共享状态形成自上而下的数据依赖链：4.1 节所设计的 MCP 统一接入层向上游提供景点元数据、行程时间矩阵与交通票务结果，是 4.2 节 RAG 检索召回与 4.3 节行程编排算法的基础数据源；4.2 节构建的双层知识增强机制向 4.3 节行程编排提供种子景点与软约束信息，并在响应生成阶段产出旅行建议与避坑提示；4.3 节给出的行程编排算法是规划链路的核心计算模块，其输出构成最终行程方案的骨架；4.4 节设计的长短期分离式记忆系统横跨上述模块，于会话起始阶段将长期偏好注入意图识别上下文，再由下游节点隐式消费；4.5 节构建的浏览器前端经由 SSE 流式通道承接上述各阶段的中间状态与最终结果，使长链路 workflows 的执行过程可被用户实时观测。

4.1 MCP 协议集成与外部工具接入

旅行规划依赖交通票务、地图 POI、目的地天气等多类异构数据源，其协议规范、参数体系与认证机制存在显著差异。为简化集成流程并增强系统可扩展性，本系统采用 MCP 协议构建外部工具与智能体间的统一接入层，针对三类外部工具分别基于 SSE 与 STDIO 两种传输模式设计客户端封装，并在调用链路中叠加重试、熔断与降级机制。

4.1.1 MCP 协议与统一客户端抽象

MCP 协议采用客户端-服务端双端架构，为屏蔽底层传输差异、保证上层调用语义一致性，本系统在客户端侧引入统一抽象层。所有具体客户端均继承自同一抽象基类，对外仅暴露以工具名称与键值对参数为输入、结构化字典为输出的异步调用接口；连接建立、消息收发、连接关闭等与传输方式强相关的逻辑则封装于子类内部，对上层完全透明。该设计将传输异构性收敛于客户端最底层，新增数据源时仅需实现对应子类，无需修改智能体节点的业务逻辑。三类客户端的封装特征对比如表 4-1 所示。

表4-1 三类 MCP 客户端的封装特征对比

客户端	服务对象	传输方式	连接形态	主要工具
AmapClient	高德地图开放平台	SSE	远程 HTTP 长连接	poi.search、poi.detail、geocode、regeo、maps_weather
TuniuClient	途牛旅游平台	STDIO	本地子进程	searchLowestPriceFlight、hotel_search 等
TrainClient	中国铁路 12306	STDIO	本地子进程	query_ticket_price

为提升系统在外部依赖故障下的鲁棒性，本系统在 MCP 客户端调用链路上构建了三层防御机制：重试层针对临时性故障采用指数退避策略，最多重试 3 次，仅对超时与可恢复错误生效；熔断层为每类客户端配置独立的三态熔断器，失败次数累计达 5 次时切换至打开状态并拒绝所有请求，60 秒后转入半开状态进行恢复探测；降级层集成于智能体节点的统一异常处理模块，遵循“局部失败、全局可用”原则，将失败字段以空列表或缺省值返回。例如，当机票数据源不可用时，交通方案节点仍可返回火车票候选结果，确保整体规划流程在单一数据源故障时仍能输出可用方案。

4.1.2 基于 SSE 的高德地图客户端

高德地图开放平台通过 SSE 协议提供地图工具的远程调用能力。由于 SSE 仅支持服务端向客户端的单向推送，MCP 协议在其上构建了非对称双通道模式：客户端通过 HTTP POST 请求发送 JSON-RPC 调用指令，服务端则通过持久化的 SSE 通道异步返回响应。该架构对客户端的请求—响应关联机制提出了较高要求。

本系统的高德地图客户端采用异步上下文管理范式组织会话流程：连接阶段依次完成 SSE 通道建立、POST 端点提取、initialize 协议协商与工具元数据缓存；工具调用阶段内部维护以请求标识符为键的待响应槽位字典，后台协程持续从 SSE 流中读取事件并写入对应槽位，主调用协程则在槽位上挂起等待。该设计将 SSE 的事件驱动模型与 Python 协程模型在语义上对齐，使远程工具调用在上层表现为普通异步函数调用。客户端为每次调用设置 30 秒超时，超时后主动释放槽位并抛出 `TimeoutError`，交由上层熔断机制处理。

4.1.3 基于 STDIO 的途牛与 12306 客户端

途牛旅游平台与 12306 铁路票务系统均以本地命令行工具形式提供 MCP 服务端，服务端进程通过标准输入读取请求、标准输出返回响应。与 SSE 模式相比，STDIO 模式无需启动 HTTP 服务、不占用网络端口，但客户端需负责服务端子进程的全生命周期管理。两类客户端在初始化阶段启动子进程，并将标准输入、输出与错误流绑定为异步管道，消息以换行符分隔按行传输，仍遵循 JSON-RPC 2.0 格式；握手与工具调用流程复用基类提供的标识符分配与槽位等待机制，仅将传输操作替换为对子进程标准输入输出流的字节读写。

两类客户端的核心差异体现在配额管理与缓存策略上。途牛平台设有严格的商业配额限制，TuniuClient 因此叠加了基于滑动窗口的频次控制器，将每分钟请求数（Requests Per Minute, RPM）限定为 5、将每日请求数（Requests Per Day, RPD）限定为 50，并

按工具粒度配置本地缓存（酒店搜索 600 秒、机票查询 180 秒），在保证数据时效性的前提下有效降低远程调用次数；TrainClient 仅提供铁路票价查询功能且无频率限制，其实现直接复用上述 STDIO 通信逻辑，仅在参数校验与响应解析环节进行轻量适配。这一设计复用从工程实践层面验证了统一客户端抽象的可扩展性。

4.2 双层 RAG 知识增强机制

大语言模型生成旅行规划内容时存在两类固有知识缺陷：一是对特定城市的细粒度旅游知识覆盖不足，易出现幻觉生成问题；二是模型训练知识存在时效边界，无法反映季节性旅游特征与最新攻略经验。为弥补上述缺陷，本系统在工作流中引入检索增强生成机制，构建结构化与非结构化协同的双层知识底座，设计了包含结构化语义分块、元数据预过滤、双单元并发抽取与多节点差异化消费的完整技术流程。

4.2.1 离线索引构建

本系统知识库语料划分为结构化与非结构化两类。结构化语料以城市为基本单元组织，存储必去景点列表、推荐路线组合等高确定性信息，系统启动时直接加载至内存字典，支持基于城市名的键值快速检索，无需进入向量化流程。非结构化语料涵盖城市攻略、酒店预订指南、实用建议与常见问题等自由格式文本，需经过完整索引流水线处理后方可使用，其内容构成如表 4-2 所示。所有语料清洗工作均前置至知识库构建阶段，由领域专家整理为规范的 Markdown 或纯文本格式，本小节重点阐述非结构化语料的索引构建方法。

表4-2 非结构化知识库的内容构成

语料类型	内容举例	主要消费环节
城市旅游攻略	目的地景点描述、玩法路线、行程经验	景点召回、行程编排
酒店预订指南	预订时机、价位区间、入住注意事项	住宿决策、信息查询
实用旅游建议	装备清单、节庆活动、季节性提示	风格适配、贴士渲染
紧急预案	突发疾病、行李遗失、证件丢失处置流程	风险提示、信息查询
常见问题	退改签政策、签证流程、票务规则等	信息查询、避坑提示

文本分块是本系统离线索引设计的核心创新点。不同于传统固定长度分块策略，本系统提出基于文档语义结构的两级分块机制。城市攻略以 Markdown 章节标题为切分边界，确保每个文本块对应一个完整主题单元；若单个章节超过预设字符阈值，则以段落为单位进行二次切分并保留少量重叠文本，避免边界信息丢失。通用文本文档则以空行分隔的自然段为聚合单位。该策略使每个文本块保持语义完整性，有效解决了固定分块导致的语义断裂与上下文混淆问题。此外，在每个文本块首部注入“城市 > 章节”形

式的层级标记，使其自带可解析的归属信息。

每个文本块在写入索引前均嵌入结构化元数据，分为来源元数据与城市元数据两类。来源元数据记录语料分类、所属城市、所属章节与文档版本，为检索阶段的精准过滤提供核心依据。城市元数据为系统预置的城市级属性，包含地理坐标、所属区域与主题标签等字段。文本块向量化由本地部署的 `bge-small-zh-v1.5` 中文嵌入模型完成，向量索引采用本地化部署的 `ChromaDB` 向量数据库承载，以余弦相似度作为度量函数，索引文件以持久化形式存储于本地磁盘，支持跨进程访问。

4.2.2 在线检索与生成

在线阶段需要在用户的自然语言请求与离线索引中以景点描述、经验叙述、风险提示等多类形态存在的攻略片段之间建立准确的语义映射，将与当前规划目标最相关的知识抽取并注入下游生成节点。直接采用标准 `RAG` 流程在本系统的场景下会面临两类典型瓶颈。一方面，用户查询往往混杂出发地、出行人数、预算等大量非语义噪声，且查询本身与目标文档在表达粒度、词汇分布与篇章形态上存在显著偏差，致使二者在向量空间中的语义距离大于其实际相关性所应反映的水平；另一方面，向量召回所依赖的双编码器架构以查询与文档独立编码为前提，运算高效却缺乏二者之间的细粒度交互建模，召回结果中位次靠前的片段未必是真正最相关者。针对上述两点，本系统在标准流程基础上引入基于假设文档嵌入的多路查询扩展与基于跨编码器的检索结果重排两项关键机制，与既有的元数据过滤、并发结构化抽取与多节点复用环节共同构成完整的在线流水线，整体结构如图 4-1 所示。

为缓解查询与文档之间的语义偏移，本系统借鉴 Gao 等^[19]提出的假设文档嵌入（`Hypothetical Document Embeddings, HyDE`）思想，将以原始查询发起检索调整为以大语言模型生成的假设文档发起检索。其依据在于，相较于用户口语化、信息混杂的自然语言查询，假设文档在词汇分布、句式结构与篇章形态上均更贴近目标语料，因而其嵌入向量在向量空间中的位置更接近真实相关片段，召回相关性得到结构性提升。具体实现层面，本系统针对三类语义目标差异显著的检索子任务设计差异化的假设文档生成模板，由大语言模型在单次结构化推理中并行生成三段假设文档，所得三组嵌入向量分别用于对应的检索路径。三类子任务的假设文档生成模板与设计取向如**错误!未找到引用源。** 所示。

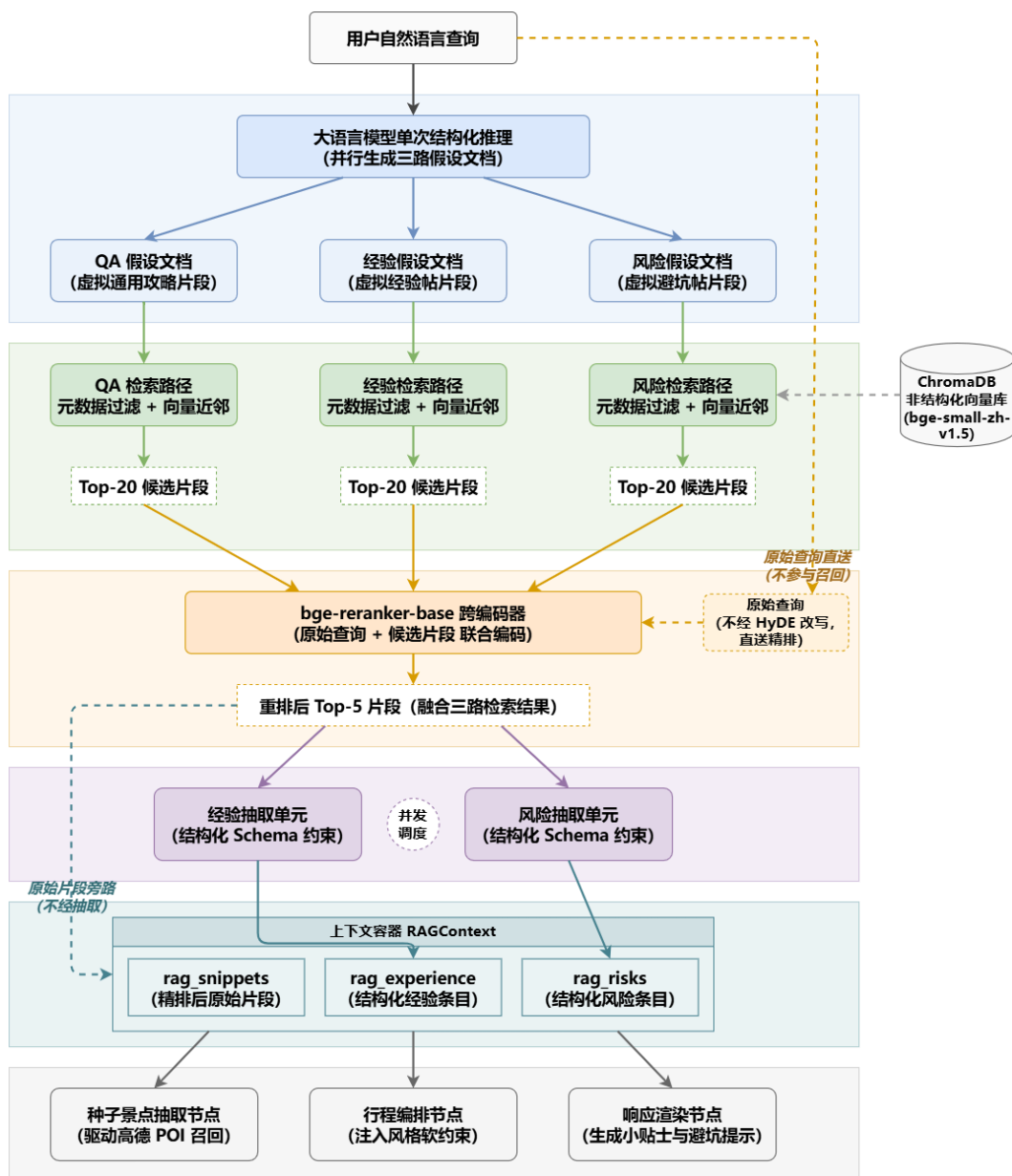


图4-1 在线检索与生成整体结构

表4-3 不同 RAG 子任务的假设文档生成模板

检索路径	假设文档生成指令	设计目的
ask-question 通用 QA	围绕 {destination} 撰写一段虚拟攻略片段， 涵盖代表性景点、典型玩法与日程节奏	使假设文档贴近通用攻略的叙述形态， 提升通用知识召回 引导嵌入向量偏向 第一人称经验叙事， 优先召回经验类片段
rag-experience 经验抽取	围绕 {destination} 与 {travel_style} 撰写一段 虚拟经验帖片段，体现可操作的行前建议与行中要点	引导嵌入向量偏向 负面与警示语义， 优先召回风险类片段
rag-risk 风险抽取	围绕 {destination} 撰写一段虚拟避坑帖片段， 覆盖常见误区、易踩雷场景与对应处置建议	

需要指出的是，假设文档自身的事实正确性并非检索阶段的关注焦点，模型即便在具体景点名称或票价上存在偏差，其整体语义信号仍能有效指引检索器在真实语料中定位与该信号最为接近的真实片段，事实层面的偏差将在后续的结构化抽取与最终生成环节由真实文档片段进行修正。三路假设文档通过单次推理并行生成的设计同时控制了 HyDE 机制引入的额外大语言模型调用开销。

每路检索内部仍沿用元数据硬过滤与向量近邻搜索相结合的混合架构。首先依据用户当前请求所涉及的城市与对应章节标签缩小候选集合，再在子集内基于假设文档的嵌入向量执行余弦相似度近邻搜索。该设计使风险类查询可直接限定于避坑指南等特定章节，避免不同城市间语义相近内容的相互干扰。针对长尾目的地元数据覆盖不足导致的召回为空问题，检索器实现了从城市与章节联合过滤，到仅城市过滤，再到全库检索的三级降级策略，确保检索链路在元数据缺失情况下仍能稳定输出。每路检索召回 Top-20 候选片段作为粗排输出，进入重排环节。

向量近邻召回所得片段虽在语义层面与查询相近，但在具体景点匹配、风险情境对齐等细粒度判别上仍存在偏差，部分片段虽位次靠前却并非真正最相关，若不加区分地全量送入下游抽取，反而会因冗余信息稀释核心约束。为此，本系统在粗排与下游抽取单元之间引入跨编码器重排环节，对粗排结果进行精排，构成召回—精排两阶段架构。与双编码器对查询与文档独立编码不同，跨编码器接收二者拼接而成的序列对作为输入，在模型内部以全局自注意力机制建模逐 token 的交互信号，从而以更高的推理代价换取更精确的相关性判定。

重排环节选用北京智源人工智能研究院开源的 bge-reranker-base 跨编码器模型，该模型与离线阶段所用的 bge-small-zh-v1.5 嵌入模型同属 BGE 系列，词表与中文表征空间高度一致，无需额外适配即可与既有嵌入流水线对接；参数量约 1.1 亿，可在单 CPU 上完成本地推理，与系统整体的本地化部署原则保持一致。具体实现层面，重排阶段以原始用户查询（而非假设文档）与每个粗排候选片段拼接为输入对，由模型联合编码后输出相关性得分，对每路检索的 Top-20 粗排结果重新排序并截取 Top-5 进入下游抽取。由于粗排与精排所承担的职能存在差异，重排端选用原始查询而非沿用假设文档。HyDE 假设文档在粗排阶段负责召回真实相关片段，而精排阶段所需判定的是该片段对实际用户需求的回应程度，使用原始查询作为衡量基准能更直接地度量片段对真实问题的相关性，避免假设文档自身的叙述偏向被进一步放大。

重排返回的片段相关性更高，但其形态仍为扁平自由文本，信息密度低且包含大量

与下游决策无关的修辞与铺垫，直接拼入推理上下文会稀释核心约束信息。本系统在抽取单元出口内置由大语言模型驱动的结构化提取模块，设置经验抽取与风险抽取两个语义互补的并行单元，通过预定义的输出 Schema 约束模型按指定字段返回结果，并辅以正反例对照引导模型保留景点名称、票价、开放时间等关键细节。两单元之间不存在数据依赖，采用并发调度同时执行，将原本串行的两次模型调用压缩至单次往返时长。两路结构化结果与精排后的原始片段一并聚合至统一上下文容器，三类产物彼此独立，支持下游节点按需读取。上下文容器填充完成后在工作流多个节点中复用，实现一次检索、多处复用的知识共享机制。种子景点抽取节点将原始片段中的具体景点名称转化为地图服务的精确检索关键词，显著提升结构化知识库未收录城市的召回效果；行程编排节点以经验条目中的风格适配信息作为软约束注入排布过程，弥补几何优化方法在个性化定制方面的不足；响应渲染节点同时读取经验与风险条目，分别生成旅行小贴士与避坑提示两个独立展示区块。三处消费节点与上下文容器之间的数据流图 4-2 所示。

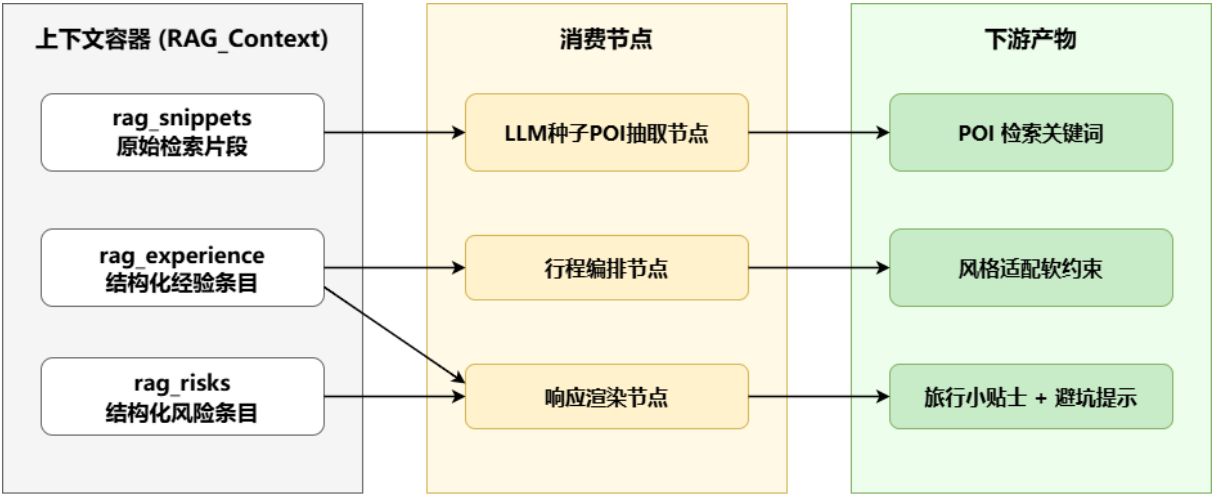


图4-2 上下文容器数据流图

4.3 行程编排算法设计

行程编排算法旨在多约束条件下生成逐日可执行的旅行方案。其输入为 P2 阶段输出的候选景点集合及用户指定的出游天数、旅行风格等参数，输出为按日组织、时段合法且移动总代价最小的景点访问序列。本节首先给出问题的形式化描述与整体算法框架，随后依次阐述三个关键子问题的求解：景点时长信息抽取与内容富化，基于行程时间矩阵的均衡 K-Means 聚类以及时段分桶与桶内旅行商问题求解。

4.3.1 问题描述与算法框架

(1) 问题的形式化描述

设候选景点集合为 $V = \{p_1, p_2, \dots, p_N\}$ 。每个景点 p_i 具有经纬度坐标 (x_i, y_i) 、用户评分 r_i 、类别标签 c_i 与最佳访问时段标签 $\sigma_i \in \{\text{上午}, \text{弹性}, \text{夜间}\}$ ，其游览时长 τ_i 由 4.3.2 节描述的抽取流程确定。用户指定出游天数 K ，由意图识别阶段抽取旅行风格 φ ；由高德 MCP 接口批量获得景点间真实行程时间矩阵 $M \in \mathbb{R}^{N \times N}$ ，其元素 $M[u][v]$ 为景点 p_u 行至 p_v 的预估耗时（分钟）。

引入决策变量 $x_{ij} \in \{0, 1\}$ 表示景点 p_i 是否被分配至第 j 日； $y_{uv}^{(j)} \in \{0, 1\}$ 表示在第 j 日的访问序列中， p_v 紧接 p_u 之后被访问。设 $T_{\max}$ 为单日活动时长上限， $s_{\min}$ 与 $s_{\max}$ 为日均景点数的双边规模参数， π 为 4.3.4 节定义的时段优先级映射。行程编排问题可形式化为如下混合整数规划：

$$\min \sum_{j=1}^K \sum_{u,v \in V} M[u][v] y_{uv}^{(j)} \quad (4.1)$$

$$\sum_{j=1}^K x_{ij} \leq 1, \quad \forall i \in \{1, \dots, N\} \quad (4.2)$$

$$s_{\min} \leq \sum_{i=1}^N x_{ij} \leq s_{\max}, \quad \forall j \in \{1, \dots, K\} \quad (4.3)$$

$$\sum_{i=1}^N \tau_i x_{ij} + \frac{1}{60} \sum_{u,v \in V} M[u][v] y_{uv}^{(j)} \leq T_{\max}, \quad \forall j \quad (4.4)$$

$$\pi(\sigma_u) \leq \pi(\sigma_v) \quad \text{whenever} \quad y_{uv}^{(j)} = 1, \quad \forall j \quad (4.5)$$

$$x_{ij}, y_{uv}^{(j)} \in \{0, 1\} \quad (4.6)$$

其中式 (4-1) 为优化目标，在语义合理的前提下最小化全行程的总移动耗时；式 (4-2) 限定每个候选景点至多被访问一次，未被访问者归入候选池余量；式 (4-3) 为容量约束，确保各日景点数落入合理区间；式 (4-4) 为单日活动时长约束，包含景点游览与跨景点移动两部分；式 (4-5) 借由时段优先级映射 π 强制访问序列服从时段先后语义。

(2) 问题性质与求解路径选择

上述问题可视为具有日均规模约束与时段顺序约束的多日 POI 路线规划问题，与带时间窗与容量约束的车辆路径问题（Capacitated Vehicle Routing Problem with Time

Windows, CVRPTW) 在结构上具有相似性。CVRPTW 已被证明属 NP-hard, 本问题作为其结构相近的变体, 其精确求解同样难以在交互式延迟约束下完成。尽管本任务中 N 处于数十量级、 $K \leq 7$, 规模并未达到不可解的程度, 联合求解仍存在两条值得审视的可行路径。

第一条路径为以元启发式方法, 以遗传算法、模拟退火、蚁群算法等为代表。其优点在于无需对问题结构进行拆解, 可在统一目标函数下处理全部约束, 但在本系统的场景下有以下三点不足: 元启发式算法的解质量对超参数较为敏感, 且通常不对解的质量提供形式化保证; 其收敛过程的不可预测性引入了显著的随机时延, 与本系统在交互式场景下对响应延迟的要求相违; 黑箱式的联合求解难以与下游审查回退机制形成可解释的耦合, 单一约束的违反将触发对整体解的重算。

第二条采用簇路由 (Cluster-First, Route-Second) 启发式, 将原问题分解为两个串行子问题: 先在地理空间内将候选景点划分为 K 个容量受控的日簇, 求解约束 (4-2) 与 (4-3); 再在每个日簇内独立求解带时段约束的访问序列, 求解约束 (4-4)、(4-5) 与目标函数 (4-1)。该求解思路有以下优点有: 第一, 分解后单日规模 m 远小于全局规模 N , 每个子问题可获得精确解或高质量近似解; 第二, 分解的粒度天然对齐下游行程审查环节的判定粒度, 重试仅需重做对应子问题而非全局重算; 第三, 模块化设计便于针对每个子问题独立选择最契合的算法并独立调试。其代价在于分解破坏了原问题的整体最优性, 所得解为联合最优解的一个上界近似。

综合权衡, 本系统选择第二条路径。考虑到旅游行程编排在用户感知上更重视语义下每日的合理性而非全程总移动时间的极致最优, 且分阶段求解所引入的次优性可经由跨日 2-opt 交换在事后阶段部分弥补, 分解策略在本任务的目标函数语义与系统响应要求下是合理的。

(3) 算法整体框架

依照上述分解思路, 本系统的行程编排算法由四个串行子模块构成: 时长信息抽取模块为每个候选景点确定 τ_i 与 σ_i (4.3.2 节); 行程时间矩阵 M 获取模块经高德接口构造 (详见 4.1.2 节); 聚类模块以景点坐标与 M 为输入将 V 划分为 K 个容量受控的日簇, 承担约束 (4-2) 与 (4-3) 的满足任务 (4.3.3 节); 时段分桶与桶内 TSP 求解模块在每个日簇内求解约束 (4-4)、(4-5) 与目标 (4-1) (4.3.4 节)。该分层设计将异质约束逐层解耦: 时段顺序约束 (4-5) 在分桶环节由结构性方式保证 (详见命题 4.1), 容量约束 (4-3) 在聚类环节由 s_{min} 、 s_{max} 双边规模参数保证, 移动代价目标 (4-1) 则分别在

跨日交换与桶内 TSP 两个尺度上逐级优化。

4.3.2 景点时长信息抽取与内容富化

游览时长与最佳访问时段是聚类与时段分桶的核心输入参数，前者直接占用每日时间预算，后者决定景点在时段桶中的归属。本节为其设计独立的抽取子流程，并配合景点富化环节为已编排的景点附加体验描述与配图。时长与时段的获取采用三级回退策略。最高优先级查询本系统离线构建的结构化城市知识库，已收录景点直接复用经人工校对的标准字段；知识库未覆盖时，调用大语言模型对剩余景点进行单次批量估算，提示词中嵌入景点体量分级与少样本示例，并在景点名后附加大类标签以提升模型对未知名景点的判断稳定性；当模型调用失败或返回非法值时，回落至预设默认值。考虑到大模型对大型景点游览时长存在系统性低估问题，本流程在结果后置阶段引入基于大类的下限校正，批量调用辅以有限次数的指数退避重试，确保单次接口抖动不会影响整体编排链路。

景点内容富化模块在已生成的编排序列基础上，为每个景点补充体验描述与配图，由两条相互独立的降级链并发执行。描述生成链优先以攻略检索片段为锚提炼简短的体验段落，未命中时改由模型基于内置知识直接生成，再失败则截取检索片段首段降级输出，最终为空串。图像获取链先经高德地图获取国内景点的高清配图，未命中时降级至 Wikipedia 图像库；查询前对景点名做后缀归一化，剥离“风景名胜区”、“景区”、“公园”等通用词以提升小众景点的命中率。两条链路均通过协程并发与流量上限调度，在控制对外部接口瞬时压力的同时显著缩短富化阶段的整体延迟。

4.3.3 基于行程时间矩阵的均衡 K-Means 聚类

景点分日聚类的本质是将候选集合中的 N 个景点划分为 K 个子集，使各子集规模与每日景点数配额相匹配。该问题具有两点显著特征，簇数 K 由用户出游天数事先指定，且每簇景点数须落入由日均景点数确定的上下界之间。下文先分析两类经典聚类方法不适用于该问题的原因，再阐述本系统的算法选择。

标准 K-Means 以最小化簇内平方误差为目标，对各簇规模没有约束。在旅游场景中，同一城市的热门景点往往在地理上高度聚集，而少数偏远景点则孤立分布。在此分布下，标准 K-Means 倾向于将密集景点分入同一簇、孤立景点单独划为一簇，造成簇规模严重失衡。在极端情况下，算法甚至会产生不含任何景点的空簇，即某一天没有景点可供安排，导致下游时段分桶与 TSP 求解因缺少输入而失效。因此，标准 K-Means 无法保证每日景点数落在合理区间内，不能满足 K 日行程日均若干景点的约束要求。

基于密度的 DBSCAN 算法同样不适用，其不足体现在两个方面。其一，DBSCAN 通过邻域半径 ε 与最小密度 minPts 间接决定簇数，簇数由数据分布形成，无法保证严格等于出游天数 K ；当形成的簇数与 K 不一致时，仍需额外进行合并或拆分处理，其逻辑复杂度并不低于直接求解约束划分问题。其二，DBSCAN 易将低密度点判定为噪声并丢弃，而偏远景点恰恰可能是长途出行的核心目的地，丢弃这些点与实际需求相悖；且当景点稀疏且分布均匀时，DBSCAN 还可能将所有景点归为单一大簇或全部判为噪声，导致聚类结果失效。此外，DBSCAN 本身不提供簇规模均衡机制，使用时仍需额外的均衡处理，叠加均衡处理后整体复杂度反而上升。

综合上述分析，在簇数 K 已知、且要求簇规模均衡的前提下，本算法选用带容量约束的均衡 K-Means 作为分日聚类方法。该算法在标准 K-Means 的迭代框架上引入 size_min 与 size_max 双边规模参数，将带容量约束的点簇分配转化为最小费用流问题求解，从机制上排除空簇与超大簇，这是该算法相对标准 K-Means 的核心改进。在距离度量上，初始划分采用对称的经纬度欧氏距离以降低批量分配开销，后续的 split/joint 提示修正、容量裁剪与跨日 2-opt 交换则改用非对称的真实行程时间矩阵作为代价函数，以更贴近实际通勤体验。三种方法在本问题上的适配性对比见表 4-4，算法的完整流程见算法 4-1。

表4-4 聚类算法在景点分日划分问题上的适配性对比

对比维度	标准 K-Means	DBSCAN	均衡 K-Means（本算法）
簇数	严格等于指定天数 K	由密度参数间接决定	严格等于指定天数 K
簇规模	无保障	无保障	$\text{size_min}/\text{size_max}$ 双边约束
空簇风险	极端分布下可能出现	低密度点全判为噪声	无
算法复杂度	$O(N)$	$O(N \log N) \sim O(N^2)$	$O(N^2)$
适用场景	无规模约束的通用聚类	簇数未知、任意形状簇	簇数已知，簇规模均衡

算法 4-1 行程时间矩阵驱动的均衡 K-Means 聚类

	Input:
	1. 兴趣点集合 $\text{POIs} = \{p_1, \dots, p_N\}$;
	2. 簇数 K (即旅行天数);
	3. 旅行风格 φ ;
	4. 行程时间矩阵 $M[N][N]$ (单位: 分钟);
	5. 合并提示集合 joint_hints ;
	6. 拆分提示集合 split_hints
	Output: 按日划分的景点簇 $G = \{G_1, \dots, G_K\}$
1	$\text{pois_per_day} \leftarrow _ \text{POIS_PER_DAY}[\varphi]$
2	$T_max \leftarrow _ \text{MAX_DAILY_HOURS}[\varphi]$

```

3  if  $K \leq 1$  or  $|\text{POIs}| \leq K$  then
4      return TRIVIAL_PARTITION(POIs, K)
5  // Step 1: 经纬度空间均衡 K-means
6  size_min  $\leftarrow \max(1, \lfloor |\text{POIs}|/K \rfloor)$ 
7  size_max  $\leftarrow \lceil |\text{POIs}|/K \rceil$ 
8  labels  $\leftarrow \text{KMeansConstrained}(K, \text{size\_min}, \text{size\_max}).\text{fit}(\text{coords})$ 
9   $G \leftarrow \text{build\_groups}(\text{POIs}, \text{labels})$ 
10 // Step 2: split-hint 修正 (违规景点迁出)
11 for each (a, b)  $\in \text{split\_hints}$  with  $\text{group}(a) = \text{group}(b)$  do
12     move b to  $\text{argmin}_{\{g \neq \text{group}(a)\}} \|\text{centroid}(g) - b\|_2$ 
13 // Step 3: joint-hint 修正 (跨簇景点合并, 允许临时超容)
14 for each (a, b)  $\in \text{joint\_hints}$  with  $\text{group}(a) \neq \text{group}(b)$  do
15     move b to  $\text{group}(a)$ 
16 // Step 4: 容量裁剪
17 dropped_pool  $\leftarrow \emptyset$ 
18 for each  $G_i$  with  $|G_i| > \text{pois\_per\_day}$  do
19     define  $\text{score}(p) := \text{rating}(p) / r_{\max} - \alpha \cdot \|p - \text{centroid}(G_i)\|_2 / \bar{d}(G_i)$ 
20     retain top-pois_per_day in  $G_i$  by  $\text{score}(\cdot)$ ;
21     dropped_pool  $\leftarrow \text{dropped\_pool} \cup (G_i \setminus \text{retained})$ 
22 // Step 5: 时间预算修正 (基于真实行程时间)
23 for each  $G_i$  do
24     for iter  $\leftarrow 1$  to 5 do
25          $h_i \leftarrow \sum_{p \in G_i} \text{estimated\_hours}(p) + \sum_{\{(u,v)\}} M[u][v] / 60$ 
26         if  $h_i \leq T_{\max}$  then break
27         while  $h_i > T_{\max}$  and  $|G_i| > 1$  do
28              $p^* \leftarrow \text{argmax}_{\{p \in G_i\}} \|p - \text{centroid}(G_i)\|_2$ 
29             remove  $p^*$  from  $G_i$ ; dropped_pool  $\leftarrow \text{dropped\_pool} \cup \{p^*\}$ 
30             update  $h_i$ 
31  $\text{FILL\_UNDERSIZED\_DAYS}(G, \text{dropped\_pool})$  // 缺额天回填
32 // Step 6: 跨日 2-opt 交换
33  $\_CROSS\_DAY\_SWAP(G, M)$ 
34 return  $G$ 

```

在计算复杂度方面, 标准 K-Means 每轮迭代只需将景点分配给最近的质心, 复杂度为 $O(N)$; 均衡 K-Means 每轮则需把带容量约束的分配转化为最小费用流问题求解, 单轮复杂度约为 $O(N^2)$, 整体高于标准 K-Means。然而复杂度上的差异在本项目上不会造成严重的响应瓶颈。旅行规划场景中出游天数 K 通常不超过一周, 每日景点数受游览节奏与体力所限多保持在数个之间, 即便计入必要的候选冗余, 待聚类景点总数 N 仍仅处于数十的量级。在此规模下, 相关子问题的求解开销均在毫秒级内, 渐近复杂度的差异并未在最终结果中得到显著体现。

K-Means 类方法对初始质心的选取较为敏感,初始化不当可能使迭代收敛至次优局部解。为应对该效应,本算法采取双重策略。首先,以十组互异的初始质心独立运行算法,并从中择取簇内距离总和最小者作为最终输出,借助随机扰动压缩单次初始化陷入劣解的概率;其次,固定随机种子,使相同输入对应于确定的聚类结果。后一策略在赋予算法可复现性的同时,亦引入一项风险,若某类城市的景点分布恰好诱发同一条次优初始化路径,算法将稳定复现该偏差而非凭借随机重试予以化解。鉴于多次重启已在统计意义上显著压缩次优解出现的概率,而确定性输出对方法的复现与算法行为的可解释性具有重要价值,该取舍在本算法适用的问题规模与场景下是合理的。值得指出的是,容量约束的引入在客观上亦缓解了算法对初始化的敏感性,双边规模约束压缩了可行分配的解空间,使不同初始化条件下迭代终态间的差异得到结构性削减。

4.3.4 时段分桶与桶内旅行商问题求解

完成分日聚类后,还需确定每一天内部景点的访问顺序。行程审查环节会检查时段是否匹配,例如夜市类景点被排进上午时段即属违规。如果日内顺序只以移动代价为优化目标,求解器有可能把夜间景点排到一天的中段,从而触发审查回环,导致整条 P3 链路重新执行。为避免这种回环,本算法把时段顺序作为硬约束,置于 TSP 求解之前。为此定义时段权重函数,将每个景点映射为其所属时段对应的权重值,具体形式如式(4-7)所示:

$$\pi(p) = \begin{cases} 0, & p \text{ 为上午类景点} \\ 1, & p \text{ 为弹性类景点} \\ 2, & p \text{ 为夜间类景点} \end{cases} \quad (4.7)$$

同一日内景点按 π 值升序划入上午、弹性、夜间三个时段桶。弹性类景点按地理就近原则并入上午桶或夜间桶;若当天只有上午景点或只有夜间景点,弹性景点全部并入该桶;若当天只有弹性景点,则单独成桶。桶间顺序由权重严格锁定,桶内顺序由 TSP 独立求解。由此从形式上保证,无论桶内顺序如何变化,按桶拼接得到的全天序列必然满足 $\pi(d_i) \leq \pi(d_{i+1})$, 从而在算法层面排除时段违规的可能。

考虑到旅行商问题为 NP-hard 问题,本算法依据桶规模 b 采用阈值切换策略:当 $b \leq 4$ 时桶内排列数至多为 $4!=24$, 直接枚举即可得到严格最优解;当 $b > 4$ 时切换为最近邻初始化结合 2-opt 局部改进的启发式方法,以 $O(n^2)$ 的单轮代价获取近似最优解。当行程时间矩阵不可用时,算法降级为欧氏距离度量,分桶机制与阈值切换逻辑完全复用。桶内 TSP 求解的伪代码如算法 4-2 所示。算法首先依时段权重将单日景点分入三

只时段桶，随后顺序遍历各桶并将桶内 TSP 的输出拼接为完整日序列，由此得到的日序列具备局部最优性与全局时段合法性。

算法 4-2 基于时段的分桶 TSP

```

Input: day_pois, transit_matrix M, city
Output: ordered_pois (单日访问序列)

1   $\pi \leftarrow \{\text{morning:0, flexible:1, evening:2}\}$ 
2  buckets  $\leftarrow \text{group\_by}(\text{day\_pois}, \text{key}=\text{p} \rightarrow \pi[\text{best\_period}(\text{p})])$ 
3  order  $\leftarrow \emptyset$ 
4  for k  $\leftarrow 0$  to 2 do
5      indices  $\leftarrow \text{buckets}[k]$ 
6      order.extend( BUCKET_TSP(indices, M) )
7  return day_pois[order]

Procedure BUCKET_TSP(indices, M):
8
9  b  $\leftarrow |\text{indices}|$ 
10 if b  $\leq 1$  then
11     return indices
12 sub  $\leftarrow \text{submatrix}(M, \text{indices})$ 
13 if M available then
14     if b  $\leq 4$  then
15         return BRUTE_FORCE(sub)
16     else
17          $\tau \leftarrow \text{NEAREST\_NEIGHBOR}(\text{sub})$ 
18         return 2OPT_IMPROVE( $\tau$ , sub)
19 else
20     points  $\leftarrow [(\text{lng}, \text{lat}) \text{ for } i \in \text{indices}]$ 
21     if b  $\leq 4$  then
22         return BRUTE_FORCE_EUCLIDEAN(points)
23     else
24         return NEAREST_NEIGHBOR_EUCLIDEAN(points)

```

与标准 TSP 相比，该算法提出的时段分桶 TSP 的核心区别在于在优化目标之外额外施加了时段语义约束。标准 TSP 仅以最小化访问序列的移动总代价为目标，并未对景点的访问时刻加以限制，因此存在将仅适宜夜间游览的景点排入上午时段的可能；所得序列虽在路径长度上最优，却违背了游览活动的时段语义，当此类违背由下游审查环节检出时，须以重新求解为代价予以修正。时段分桶 TSP 则先按时段权重将单日景

点划入三个有序桶，桶间顺序由权重结构性锁定，求解被限定于桶内进行：桶内移动代价的局部最优性与全天序列的时段合法性由此同时得到保证。就计算复杂度而言，标准 TSP 的精确求解为 $O(n!)$ 、启发式求解为 $O(n^2)$ ，且启发式解与最优解之间通常存在 15% 至 25% 的偏差；引入分桶后，单桶规模 m 远小于全天景点数 n ，桶内精确枚举的代价由 $O(n!)$ 降至 $O(m!)$ ，启发式求解的规模亦相应收缩。两种 TSP 方法的对比见表 4-5。

表4-5 标准全局 TSP 与时段分桶 TSP 的对比

对比维度	标准全局 TSP	时段分桶 TSP（本算法）
优化目标	移动总代价最短	时段约束下的移动代价最短
时段合理性	无保障	桶间顺序结构性锁定
问题规模	全日 n 个景点	单桶 m 个景点， m 远小于 n
求解复杂度	$O(n!)$ 精确 / $O(n^2)$ 启发式	单桶 $O(m!)$ 精确 / $O(m^2)$ 启发式
最优性	全局最优或全局近似最优	桶内最优，桶间顺序不可比
适用场景	纯路线长度优化	体验导向、时段语义重要的行程

4.4 记忆系统设计

旅行规划场景下的记忆需求具有双重特征：单会话内需维持多轮交互上下文以支撑需求澄清与方案修正，跨会话则需沉淀用户长期偏好以规避重复性询问。若将两类需求置于同一存储介质，不仅会提升状态读写的逻辑复杂度，还会因偏好条目与临时上下文相互干扰导致规划质量下降。本系统的记忆模块采用长短期记忆严格分离的设计范式，由差异化存储机制分别承载两类数据，并通过统一入口对外提供访问服务。

4.4.1 记忆分层架构与统一访问入口

记忆系统采用短期会话上下文与长期用户偏好的双层架构，短期层存储当前会话的消息序列与图状态中间结果，其生命周期与会话标识 `thread_id` 绑定；长期层存储跨会话的用户画像、偏好条目与历史行程，其生命周期与用户标识 `user_id` 绑定。两层之间无直接数据通路，仅在工作流节点执行时通过显式调用将长期记忆的关键信息注入当前会话提示词。系统对外访问由 `MemoryManager` 类统一封装，内部通过组合 `LongTermMemory` 实例屏蔽底层存储差异。其核心方法 `summarize_for_prompt` 在每轮会话启动时，将长期偏好与近期历史行程序列化为单条 `SystemMessage` 并置于消息序列头部，由 P1 阶段意图识别节点注入大语言模型上下文；为抑制上下文膨胀，序列化结果长度上限设定为 500 字符。该方法实现了记忆存储与使用的解耦，功能节点无需直接

操作底层文件或数据库。

4.4.2 长短期记忆的存储实现

短期会话上下文以 LangGraph 内置的 Checkpointer 作为唯一数据源, Web 端使用 AsyncSqliteSaver 将状态持久化至 SQLite 文件 data/aligo.sqlite。当请求进入工作流时, 框架依据 thread_id 自动加载对应会话的状态快照, 节点执行完成后将更新后的状态写回数据库, 整个过程对节点函数透明。长期记忆由 LongTermMemory 类管理, 以 JSON 文件形式按用户维度独立存储于 data/memory/{user_id}.json 路径, 文件内部通过 preferences、trip_history、chat_history 与 statistics 四个字段, 分别存储偏好条目列表、历史行程摘要、跨会话聊天摘要及由前述字段实时聚合生成的统计指标(如高频目的地、平均预算)。两类存储在物理层面完全隔离, 任一方的损坏或清空不会影响另一方, 同时便于在调试与隐私清理场景下对单一维度数据进行定向操作。

4.4.3 偏好的自动沉淀与上下文注入

为降低用户显式记忆维护成本, 本系统在意图识别节点中集成偏好隐式提取机制, 要求大语言模型在输出意图类型与关键实体的同一次推理调用中, 从用户自然语言表述中抽取隐含偏好信息(如“我们家一般住汉庭”蕴含的酒店品牌偏好、“尽量坐高铁不坐飞机”蕴含的交通方式偏好), 并在节点函数中立即调用 LongTermMemory 的写入接口完成持久化。该设计将意图理解与偏好提取合并为单次推理请求, 相较于独立调用模式, 每轮对话可节省约一次模型请求的延迟与 token 开销。

长期偏好经 4.4.1 节所述机制以系统消息形式回注当前会话上下文后, 下游节点可隐式调用其中字段: 住宿查询节点优先检索用户偏好的酒店品牌, 交通查询节点优先按用户偏好的航司与座位等级排序, POI 检索节点借助高频目的地推断当前出行的兴趣类型。整个过程无需用户在每轮对话中重复声明, 实现了从被动询问到主动推断的交互模式转变。

4.5 前端设计与交互展示

为使前述工作流与算法以友好的形式触达终端用户, 本系统在后端服务之上构建了基于浏览器的交互式前端。前端不仅承担自然语言输入与行程结果展示的基本职能, 还需在长链路工作流执行过程中提供清晰的进度反馈与中间状态呈现。本节依次介绍前端的技术栈选型与整体架构、主要界面的组件设计, 以及面向长耗时工作流的流式响应渲染机制。

4.5.1 前端技术栈与整体架构

前端采用单页应用（Single Page Application, SPA）架构，以 Vite 为构建工具、React 为应用框架。Vite 基于原生 ES 模块实现按需编译，在冷启动与热更新速度上具有显著优势，适配前端高频迭代开发需求。React 以函数式组件、Hooks 状态管理与单向数据流为核心特性，能够高效支撑对话式应用中频繁的局部状态更新与列表渲染；其成熟的生态系统使样式、路由与状态管理等能力可通过模块化方式集成，有效降低了开发与维护成本。

前后端通过清晰的接口边界实现解耦。后端 FastAPI 服务提供两组 REST 路由，分别负责会话管理与对话流式响应。用户首次进入系统时，前端调用会话创建接口获取唯一 session_id 作为本次会话句柄；后续所有用户输入均通过 SSE 方式触发对话路由，前端以 EventSource 客户端持续接收后端推送的状态变更与文本增量，直至响应流结束。相较于传统一次性请求 - 响应模式，该通信机制可在 workflow 未完成时持续渲染中间结果，显著缓解用户等待焦虑。

前后端协同时序如图 4-3 所示。用户输入自然语言需求后，前端先验证会话句柄有效性，再通过 POST 请求向对话端点提交消息体并立即进入 SSE 监听状态；后端接收请求后将消息注入 LangGraph 工作流，每个节点执行完成后通过 yield 机制向前端推送阶段性事件，前端根据事件类型增量更新界面；当响应渲染节点生成最终行程后，后端发送结束标志，前端关闭 SSE 连接并将完整结果固化为可回溯的会话历史条目。

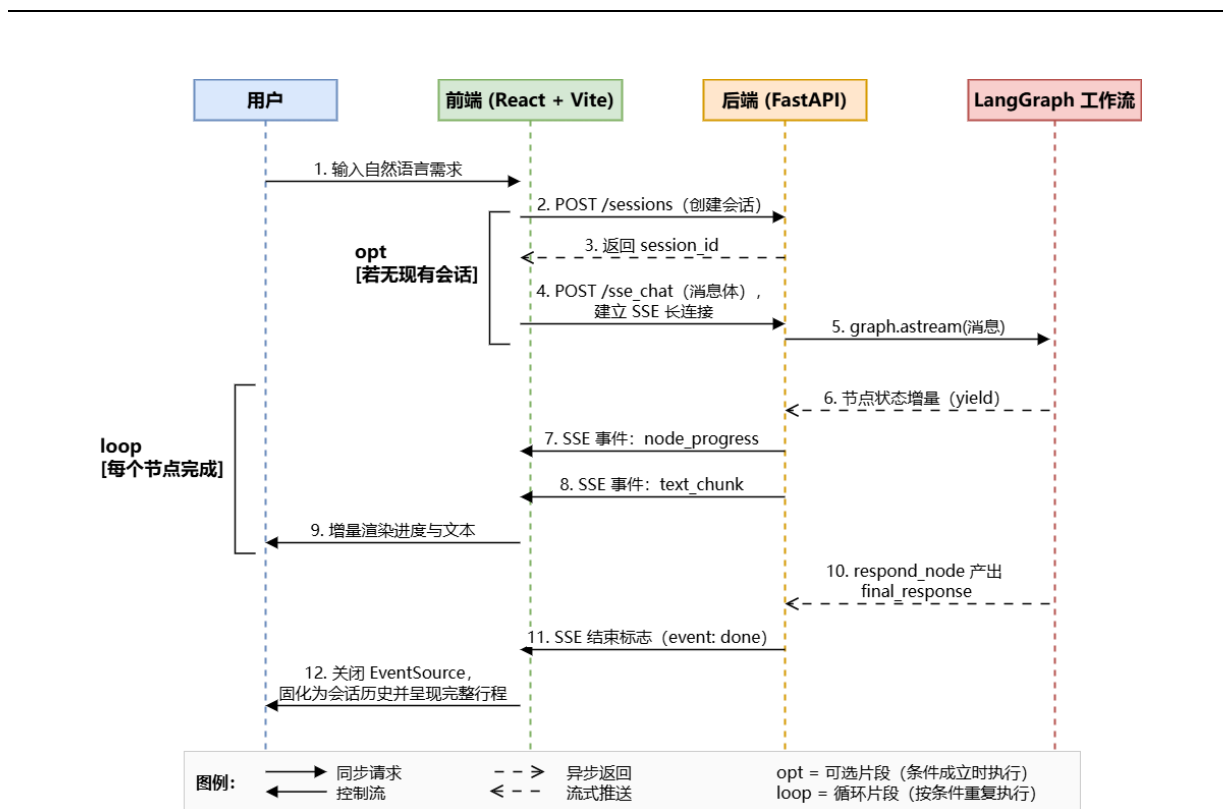


图4-3 前后端协同时序

4.5.2 主要界面设计

系统首先呈现登录界面，用于隔离不同用户的长期记忆与偏好数据（图 4-4）。登录后进入主界面，采用左右双区域布局：左侧为会话列表与历史检索区，右侧为对话主区（图 4-5）。该布局遵循主流对话式应用的视觉范式，可在不增加用户学习成本的前提下，覆盖多会话切换、历史查阅与新会话创建等高频操作。

对话主界面是用户与系统交互的核心入口。区域顶部固定显示当前会话标题与状态指示器，中部为可滚动消息流，底部为输入框与发送按钮。消息流采用气泡式设计区分用户消息与系统回复：用户消息右对齐并使用主题色填充，系统回复左对齐并以浅色背景增强可读性。对于流式生成中的系统回复，气泡末端添加光标动画以传达正在处理的状态。对话主界面的运行效果如图 4-6 所示。

行程结果展示页是系统核心输出的呈现载体。考虑到完整行程包含交通方案、逐日景点安排、住宿、餐饮与预算等多维度信息，单一视图难以兼顾信息密度与可读性，前端将其拆分为概览卡片、逐日时间线、地图视图与预算明细四个模块。概览卡片以单屏可见的形式展示行程名称、总天数、出行日期与参与人数；逐日时间线按日划分，每日内部依次列出景点、餐饮与住宿条目，并标注预估访问时长与节点间通勤时间；地图视图基于高德地图 API，将每日景点序列以折线连接渲染，并通过颜色编码区分不同日期路线。行程结果展示页整体形态如图 4-7、图 4-8 所示。



图4-4 TripGo 登录界面

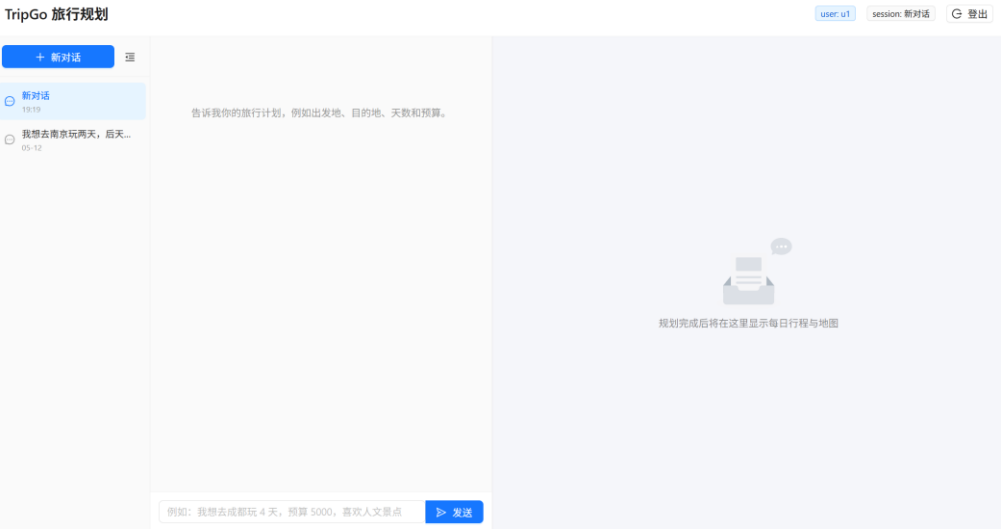


图4-5 TripGo 主要界面

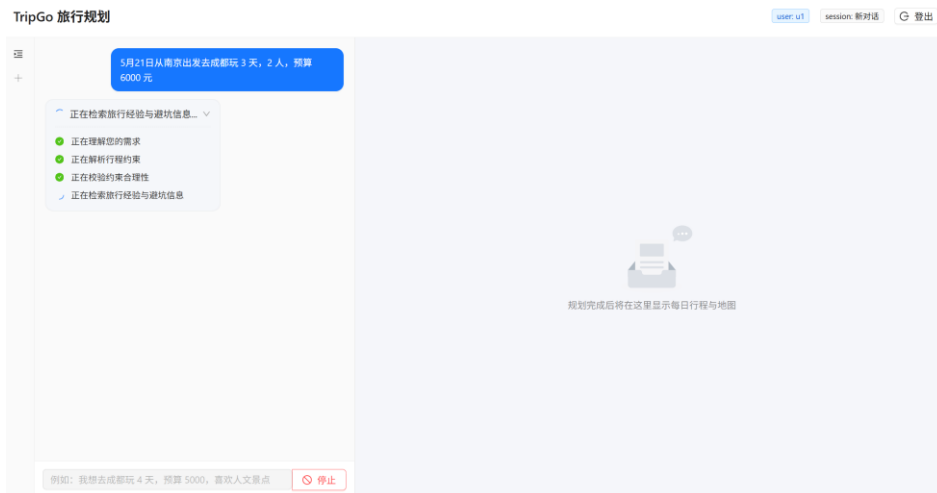


图4-6 TripGo 对话主界面

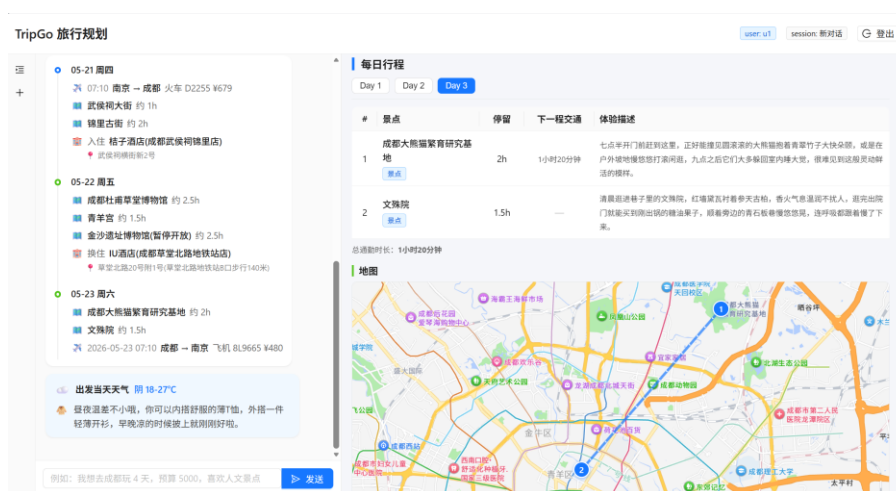


图4-7 行程结果展示页（a）

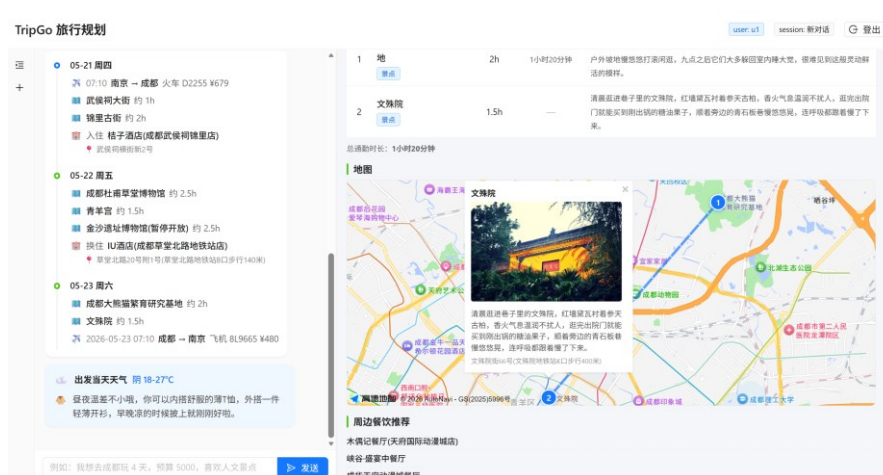


图4-8 行程结果展示页（b）

交通方案对比页用于呈现往返程候选交通工具。各方案以行形式展示出行方式、出发到达时间、票价与系统推荐标记；用户可通过列头排序按时间或票价快速筛选。系统依据全局最低票价标记的推荐项以图标高亮显示，便于用户快速定位最优选择。交通方案对比页运行效果如图 4-9 所示。



图4-9 交通方案对比页

4.5.3 流式响应渲染

旅行规划 workflow 端到端执行时间通常为数秒至数十秒，若界面保持静默直至最终结果返回，将导致用户感知出现明显等待空窗。为弥合后端长链路计算与前端瞬时反馈之间的体验差距，本系统在工作流执行边界与前端视图层之间构建了基于 SSE 的流式渲染通道。

后端侧以异步生成器为核心实现。对话路由处理函数将 LangGraph 的 `astream` 接口封装为异步迭代器，依次提取工作流执行过程中的状态增量；每当一个节点执行完成，路由函数即将节点名称、本次更新字段子集与可读进度描述封装为 JSON 事件帧，通过 `yield` 关键字交由 FastAPI 的 `StreamingResponse` 推送至客户端。前端侧基于浏览器原生 `EventSource` API 实现，构造事件类型路由器对三类事件分别处理：节点级进度事件触发顶部状态指示器更新，将 "正在查询交通方案" "正在编排行程" 等阶段性文案逐步展示给用户，使工作流执行过程具备可观测性；文本增量事件追加至当前生成中的消息气泡，配合 `React` 并发渲染机制实现逐字呈现的打字机效果，将长文输出的等待过程拆解为可感知的连续动作；终止事件触发会话历史持久化与状态指示器复位。

为应对网络抖动或后端节点异常等场景，前端在 `EventSource` 之上叠加了重连与状

态回滚机制。当连接因网络原因断开时，前端按照指数退避策略在有限次数内尝试重连，并复用 `session_id` 请求后端续传未推送内容；当后端节点异常导致 workflow 中止时，前端将当前消息气泡标记为失败状态并提供重试入口，用户可在不丢失上下文的前提下重新触发 workflow。流式渲染与重连回滚机制的协同，使前端在处理长链路 workflow 时既具备良好的实时反馈能力，又具备一定的容错韧性。

4.6 本章小结

本章基于前文的总体设计，详细阐述了旅行规划智能体各核心模块的设计理念与工程实现。在 MCP 协议集成与外部工具接入方面，设计了统一的客户端抽象层，并分别基于 SSE 和 STDIO 传输方式封装了高德地图、途牛与 12306 三类异构服务，集成了重试、熔断与降级机制以保障调用韧性。针对双层 RAG 知识增强机制，提出了结构化知识库与向量库协同的架构，引入了假设文档嵌入和跨编码器重排来提升检索相关性，并设计了并发结构化抽取与多节点复用流程。行程编排算法是本系统的计算核心，其将问题分解为景点分日聚类与日内路径求解，分别采用带容量约束的均衡 K-Means 算法和基于时段分桶的 TSP 算法，实现了大语言模型语义理解与传统运筹优化算法的高效协同。记忆系统采用长短期分离的架构，通过单次推理合并完成意图识别与偏好抽取，并实现跨会话的自动注入。此外，本章也简要介绍了前端的 SPA 架构、主要界面与基于 SSE 的流式响应渲染机制，以提供完整的用户交互体验。各模块通过精心设计的状态依赖链协同工作，共同支撑起端到端的规划能力。

第五章 旅行规划智能体系统测试

为验证所设计系统的功能完整性、性能可行性与架构有效性，本节在统一环境下依次开展端到端功能测试、对比测试与消融实验。端到端功能测试以工作流的主要分支为单位构造测试用例，检验意图识别、约束抽取与校验、并行数据获取、行程编排与预算自检等关键节点能否按设计协同产出预期结果，并测量端到端时延与各阶段耗时等性能指标；对比测试以单次调用的纯大语言模型方案作为基线，在统一评测集上与本系统逐指标对照，量化所提架构相对于直接调用大语言模型的实质性增益；消融实验在对比测试评测集上对系统内部各核心模块进行单变量替换，进一步将整体性能差距分摊至各模块的独立贡献。三组实验共享 5.1 节所述的测试环境与系统配置，测试方法见 5.2 节，结果与分析分别见 5.3 节至 5.5 节。

5.1 测试环境与系统配置

功能测试与对比测试均在同一单机环境下进行。硬件配置：Intel Core i9-14900HX CPU，32GB 内存；软件配置：Windows 11 操作系统，Python 3.11，Node.js 20.10；核心依赖包括 LangGraph 0.2、FastAPI 0.110、langchain-openai 0.2 等，向量数据库采用 ChromaDB 嵌入式模式部署。

大语言模型选用火山引擎 doubao-seed-2-0-lite 256K 接口无思考模型，上下文窗口上限 256K，单次响应 max_tokens 配置为 8192。为平衡结构化输出稳定性与生成内容多样性，温度参数（temperature）按节点类型差异化设置（表 5-1）：意图识别与约束抽取节点要求严格输出符合 JSON Schema 的字段集合，采用较低温度抑制随机性；行程编排与响应渲染节点需体现适度风格化表达，采用中等温度；RAG 经验抽取与种子景点提取节点介于两者之间，以在事实保真度与表达多样性之间取得最优平衡。

针对长会话上下文膨胀风险，系统实施两处长度控制：长期记忆注入的系统消息按 4.4.1 节约束不超过 500 字符；RAG 检索片段按相似度排序取 Top-5 后截断，单片段长度不超过 800 字符。

表5-1 各类型节点的温度参数配置

节点类型	代表节点	温度
结构化输出	意图识别、约束抽取	0.2
知识抽取	RAG 经验、RAG 风险、种子景点提取	0.4
创造性生成	行程编排、响应渲染	0.7

对比实验与消融实验中，基线组采用与本系统一致的底层模型及采样温度。二者的区别仅在于本系统在模型之外叠加了状态图编排、MCP 工具接入与双层检索增强，而基线不含上述组件。上述配置使两种方案的性能差异完全地归因于架构要素本身，而非模型能力的差异，从而保证对比结论的可信度。

5.2 测试方法

本章测试由端到端功能测试、对比测试与消融实验三组实验组成，三者测试粒度与目的上层层递进。端到端功能测试以工作流的主要分支为单位选取四组代表性用例，从功能链路完整性、阶段耗时与外部接口可用性三个层面检验系统能否按设计协同输出预期结果。对比测试在扩展后的 12 条评测集上将本系统与纯大语言模型基线逐指标对照，量化所提架构在结果质量上的整体差异。消融实验沿用对比测试的评测集，对核心模块依次执行退化替换，将整体差距分摊至各模块的独立贡献。三组实验在评测集上呈嵌套关系：端到端功能测试的四组用例为对比测试评测集中的代表性子集，用于细粒度展示工作流执行过程；对比测试与消融实验在更大评测集上进行统计性验证。各组实验的具体方法、用例与指标定义分别在 5.3、5.4 与 5.5 节首段给出。

5.3 端到端功能测试

端到端功能测试从功能、性能与稳定性三个层面对工作流进行综合检验。功能层面验证各阶段节点能否按设计协同输出预期结果，性能层面记录端到端响应时延、各阶段耗时与外部接口调用次数等量化指标，稳定性层面对各用例重复执行 5 次，检验工作流在外部接口偶发抖动条件下的局部降级能力。为覆盖工作流主要分支，本节设计四组测试用例，分别对应知识库覆盖城市典型规划、非知识库城市兜底召回、约束缺失触发对话式协商以及非规划意图触发偏好沉淀。其中前两组用例的目的地选自 5.4 节对比测试评测集（用例一对应 E01，用例二对应 E07），在主干评测集上以细粒度方式展示工作流执行过程，与 5.4 节在更大评测集上的统计性验证形成互补。

用例一：典型知识库城市规划

输入：“5 月 21 日从南京出发去成都玩 3 天，2 人，预算 6000 元”。用例细节及结果见表 5-2。意图识别节点判定为规划意图，约束抽取节点完整解析出发地、目的地、起止日期、人数与预算五项硬约束。并行数据获取阶段同时返回往返交通候选 12 条、RAG 经验条目 8 条、避坑提示 5 条。POI 召回阶段命中知识库必去清单与推荐组合，

返回景点 23 处。行程编排阶段将入选景点分配至 3 日并求解最优访问序列。预算自检阶段判定交通与住宿合计占总预算 58%，未触发降档逻辑。端到端输出包含完整逐日行程与住宿推荐，结果见图 5-1、图 5-2、图 5-3。

表5-2 用例一测试用例表

用例编号	测试功能	输入条件	预期结果	实际结果	是否通过
5 月 21 日从南京出发					
TC-01-1	意图识别	去成都玩 3 天，2 人，预算 6000 元	识别为规划意图	正确识别为 planning	通过
TC-01-2	约束抽取	同上	解析出 5 项硬约束	5 项硬约束完整解析	通过
TC-01-3	并行数据获取	同上	RAG 与往返交通三路并发	交通 12 条、经验 8 条、风险 5 条	通过
TC-01-4	POI 召回	destination=成都	命中知识库必去清单	召回有效景点 23 处	通过
TC-01-5	行程编排	23 个 POI、3 天	聚类与 TSP 求解	3 日有序景点序列	通过
TC-01-6	预算自检	总预算 6000 元	占比低于阈值不降档	占比 58%，未降档	通过



图5-1 用例 1 结果展示（a）

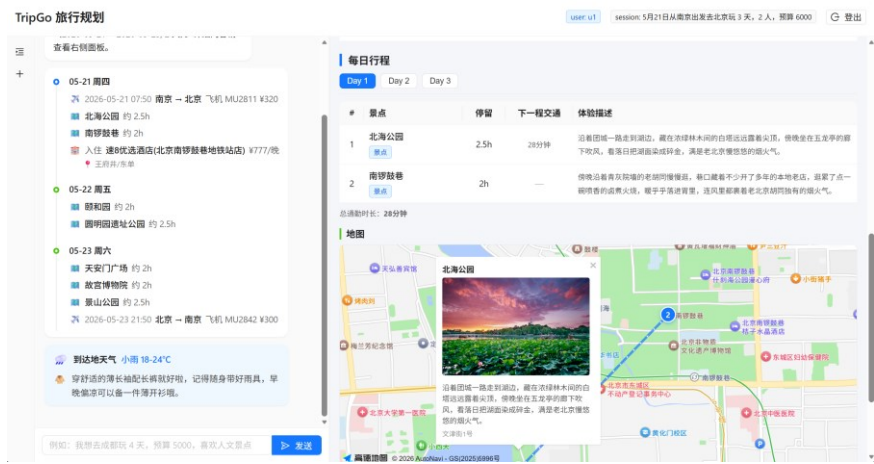


图5-2 用例 1 结果展示（b）

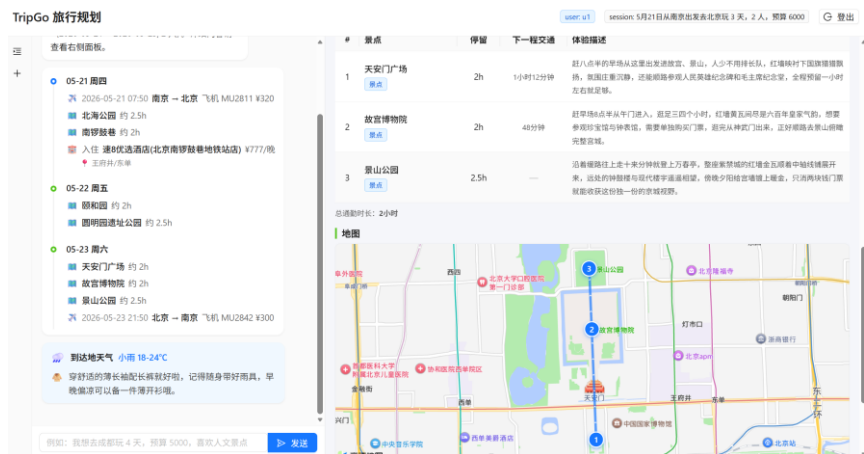


图5-3 用例 1 结果展示（c）

用例二：非知识库城市兜底

输入：“5 月 24 日从南京出发去黄山玩 3 天，3 人，预算 5000 元”。用例细节及结果见表 5-3。由于黄山未收录于结构化城市知识库，POI 召回阶段无法依赖必去清单与推荐组合，转而由 LLM 种子提取节点生成 12 个具体景点名称，连同攻略关键词一并提交高德地图 POI 接口检索，最终召回有效景点 6 处。行程编排与后续节点正常执行，端到端输出可执行行程。由此可见，在知识库未覆盖目的地下降级路径仍能产出可执行行程。结果见图 5-4、图 5-5。

表5-3 用例二测试用例表

用例编号	测试功能	输入条件	预期结果	实际结果	是否通过
TC-02-1	意图识别	5 月 24 日从南京出发 去黄山玩 3 天，3 人，预算 5000 元	识别为规划意图	正确识别为 planning	通过
TC-02-2	知识库匹配	destination=黄山	未收录则触发降级路径	降级至种子提取	通过
TC-02-3	LLM 种子提取	RAG 检索片段	输出具体景点名清单	提取 12 个景点名	通过
TC-02-4	高德 POI 检索	种子+关键词提示	返回有效景点	召回 6 处有效景点	通过
TC-02-5	端到端输出	前述结果汇总	可执行 3 日行程	正常输出完整行程	通过



图5-4 用例 2 结果展示（a）

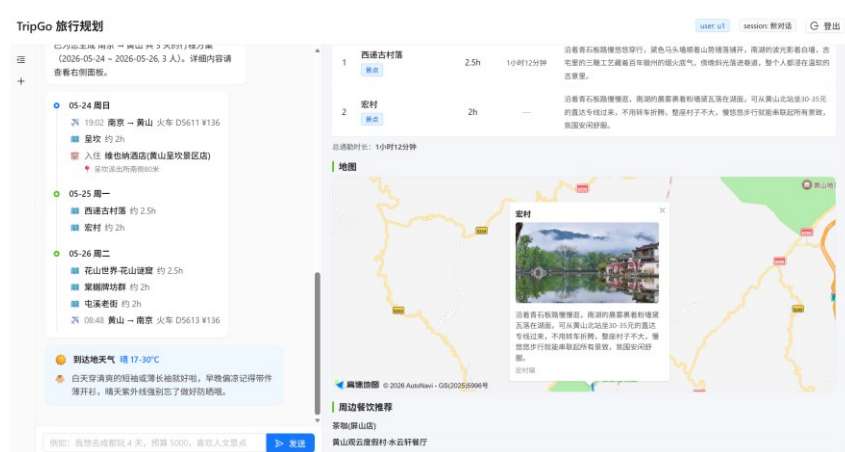


图5-5 用例 2 结果展示（b）

用例三：约束缺失协商路径

输入：“5 月 24 日出发去黄山玩 3 天，3 人，预算 5000 元”，相较用例二缺失出发地硬约束。用例细节及结果见表 5-4。意图识别节点仍正确判定为规划意图，约束抽取节点解析出目的地、起止日期与预算后将出发地标记为空。约束校验节点检测到关键级别违例并写入 `rule_violations`，条件路由函数依据违例级别将控制流导向协商节点，由协商节点根据缺失字段生成自然语言追问话术。整个流程在校验阶段提前终止，未触发后续并行数据获取与行程编排，避免了在信息不全情况下产出低质量结果。本用例对应约束不完整场景下的协商分支，结果显示系统能够在校验阶段及时早停并触发追问。结果见图 5-6。

表5-4 用例三测试用例表

用例编号	测试功能	输入条件	预期结果	实际结果	是否通过
TC-03-1	意图识别	5 月 24 日出发去黄山玩 3 天，3 人，预算 5000 元（缺出发地）	识别为规划意图	正确识别为 <code>planning</code>	通过
TC-03-2	约束抽取	同上	出发地标记为空	<code>origin</code> 字段为 <code>None</code>	通过

TC-03-3	约束校验	rule_violations 检测	检出 critical 违例	违例正确写入	通过
TC-03-4	条件路由	违例级别=critical	导向协商节点	路由至 negotiate	通过
TC-03-5	协商话术	缺失字段=origin	生成自然语言追问	追问话术生成	通过
TC-03-6	早停验证	校验失败	后续阶段不执行	P2 至 P5 未触发	通过



图5-6 用例 3 结果展示

用例四：非规划意图偏好沉淀

输入：“我喜欢住全季”。用例细节及结果见表 5-5。意图识别节点判定为偏好更新意图，在同一次推理调用中从关键实体字段提取出酒店品牌 "全季", 并写入酒店品牌偏好列表。条件路由函数依据意图类型将控制流分发至偏好确认节点，生成简短回执消息后结束本轮工作流。本用例反映了意图分流机制对非规划场景的正确识别，以及偏好提取与意图理解在单次推理中的融合。在下一轮规划请求中，该偏好被拼入系统消息，住宿节点在筛选候选酒店时优先匹配该品牌，体现了长期记忆与下游节点之间的协同链路。结果见图 5-7。

表5-5 用例四测试用例表

用例编号	测试功能	输入条件	预期结果	实际结果	是否通过
TC-04-1	意图识别	我喜欢住全季	识别为偏好更新意图	识别为 preference_only	通过
TC-04-2	偏好抽取	同次 LLM 推理调用	抽取酒店品牌字段	提取出全季	通过
TC-04-3	长期记忆写入	hotel_brand=["全季"]	持久化至 JSON 文件	写入成功	通过
TC-04-4	回执生成	偏好已记录	生成简短确认回执	确认消息已生成	通过
TC-04-5	跨会话注入	下一轮规划请求	系统消息含全季偏好	偏好被正确注入	通过

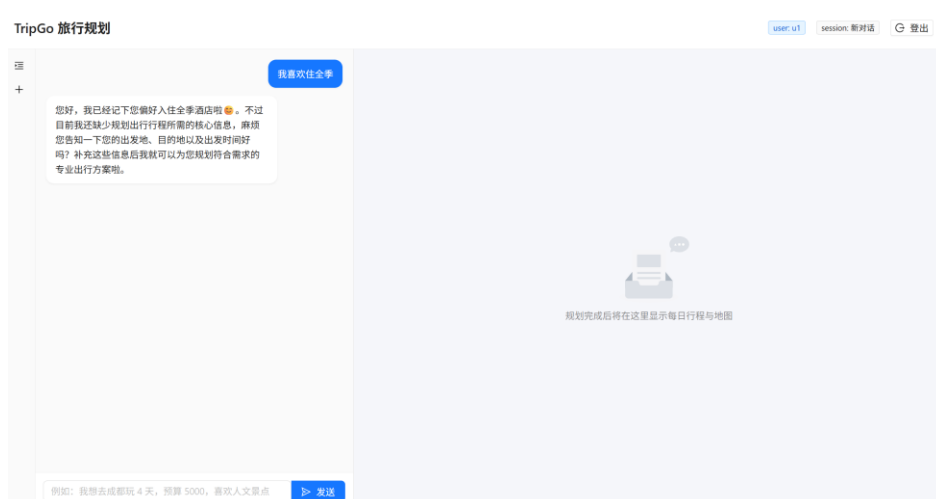


图5-7 用例 4 结果展示

上述四组用例各重复执行 5 次，对端到端时延、各阶段耗时与外部接口调用次数取均值，统计结果如表 5-6 所示。

表5-6 关键性能指标统计

指标	用例一	用例二	用例三	用例四
端到端时延（秒）	79.6	84.3	13.5	11.8
P2 并行数据获取耗时（秒）	12.3	20.2	/	/
P3 行程编排耗时（秒）	42.6	43.1	/	/
P4 住宿与预算耗时（秒）	11.3	13.6	/	/
LLM 调用次数	7	9	3	4
MCP 工具调用次数	18	21	0	0
MCP 调用成功率（%）	100	100	/	/

由表 5-6 可见，典型规划场景端到端时延约 80 秒，结合 SSE 流式渲染的逐段反馈机制，用户实际感知等待时间显著小于该绝对值。P2 阶段的并行扇出将原本串行的三路数据获取压缩为三者最大值，有效缩短了端到端关键路径。MCP 调用成功率在用例一与用例二中均达 100%，结合熔断与降级机制，单一外部接口偶发失败未造成全局阻断，系统在所选用例上具备容错能力。

5.4 对比测试

对比测试以单次调用的纯大语言模型方案为基线，从架构层面量化所提系统相对于直接调用大语言模型的增益。基线由 5.1 节所述的同一模型配合一段结构化提示词单次调用完成行程生成；该提示词包含一条 one-shot 示例，要求模型输出涵盖交通班次与票价、住宿、餐饮及逐日预算的逐日行程。本系统与基线在同一组评测用例上各独立运行 5 次并取均值。评测从行程质量与系统性能两个层面选取七项指标：行程质量层面

包括 POI 有效率、POI 幻觉数、日均景点间移动耗时、时段违规数、预算可核算且不超过支比例与交通住宿信息真实性六项；系统性能层面取端到端时延一项。其中 POI 有效率与幻觉数共同刻画景点信息真实性，移动耗时反映行程地理合理性，时段违规数与预算指标按明文规则判定，结果可复现。

对比测试所采用的评测集包含 12 条用例，如图 5-7 所示。所有用例采用统一查询模板“{出发日期}从南京出发去{目的地}玩{天数}天，{人数}人，预算{预算}元”，旅行风格则以“想深度体验”、“带小孩”、“想多打卡”等关键词在自然语言中隐式表达，以贴近真实用户的表述习惯。评测集中知识库收录城市与未收录城市各 6 条，用于检验双层检索增强在长尾城市上的容错能力。行程天数跨越 2 至 5 天，旅行风格涵盖深度、打卡、亲子、情侣四类各 3 条。此外，评测集引入 E11 与 E12 两条人均日预算最少的用例用于针对性检验预算自检与降档回环机制。本系统与纯 LLM 基线在上述评测集上的对比结果如表 5-8 所示。

表5-7 对比实验评测集

编号	目的地	知识库	天数	人数	预算(元)	人均日预算	风格
E01	成都	收录	3	2	6000	1000	深度
E02	北京	收录	5	2	12000	1200	深度
E03	西安	收录	3	4	5000	417	深度
E04	杭州	收录	2	2	1600	400	亲子
E05	重庆	收录	4	3	7000	583	亲子
E06	上海	收录	3	2	8000	1333	亲子
E07	黄山	未收录	3	3	5000	556	打卡
E08	厦门	未收录	4	3	5000	417	打卡
E09	青岛	未收录	3	4	5000	417	打卡
E10	大理	未收录	5	2	7000	700	情侣
E11	桂林	未收录	3	3	2400	267	情侣
E12	张家界	未收录	4	4	4000	250	情侣

表5-8 纯 LLM 基线与本系统对比实验结果

指标	纯 LLM 基线	本系统
POI 有效率(%)	78.0	99.0
POI 幻觉数(个/方案)	1.6	0.1
日均景点间移动耗时(分钟)	156	98
时段违规数(个/方案)	1.3	0.05
预算可核算且不超过支比例(%)	63	92
交通与住宿信息真实性(%)	16	96
端到端时延(秒)	16.2	88.5

由表 5-8 可见，两种方案在多数指标上存在显著差距。在 POI 有效率、POI 幻觉数、时段违规数与交通住宿信息真实性四项指标上，二者差距达到数量级水平。预算可核算且不超支比例亦得到明显提升，本系统残余的未达标用例集中于 E11、E12 两条人均日预算最紧的情形，表明在预算极度受限时即便经过降档回环仍可能不存在可行解。端到端时延是基线略占优势的指标，本系统因多阶段编排、多次模型调用与多次 MCP 往返，端到端耗时显著高于单次调用的基线，该结果与 5.3 节所测得的时延量级一致。然而考虑到旅行规划属于低频、决策成本较高的任务，结果可执行性通常优先于响应速度，因此该差距在本任务场景下处于可接受范围。综合上述指标，所提架构在旅行规划任务上相对于直接调用大语言模型具备明显的结果质量优势。

5.5 消融实验

5.4 节的对比测试从整体层面反映了本系统相对于纯大语言模型基线的差距，但该结果是全部模块协同作用下的综合效果，无法揭示各核心模块对整体性能的独立贡献。为进一步量化系统内部关键模块的作用、明确各项设计的有效性来源，本节沿用 5.4 节的评测集并遵循单变量原则，每次仅将一个目标模块替换为功能最朴素的退化版本，其余模块与运行配置保持不变，由此将性能差异归因于被替换模块本身。退化版本并非直接删除对应节点，而是以一种最基础的实现予以替代，使其在功能上仅维持流程连通而不具备原有的优化能力。每一消融配置在每条用例上重复运行 5 次并取均值。本节针对系统的四个核心模块设计相应的消融配置，依次涵盖 RAG 知识增强、均衡 K-Means 容量约束、时段分桶机制、桶内 TSP 与跨日 2-opt 路径优化，各配置的退化方式与主要观测指标如表 5-9 所示。编排与知识增强相关模块的消融结果如表 5-10 所示。

表5-9 消融实验配置

编号	消融模块	退化版本	主要观测指标	重点用例
A1	RAG 知识增强	跳过检索节点， 种子景点提取不依赖检索片段	POI 幻觉数、 非知识库城市召回数	E07-E12
A2	均衡 K-Means 容量约束	以标准 K-Means 替代， 取消簇容量上下界	簇大小标准差、 容量违规率	全部用例
A3	时段分桶机制	取消三时段桶划分， 单日景点直接全局求解	时段违规数、 审查重试次数	全部用例
A4	桶内 TSP 与 跨日 2-opt	取消路径优化， 簇内景点按评分降序排列	日内景点间移动耗时	全部用例

表5-10 编排与知识增强模块消融结果

编号	POI 幻觉数	非知识库城市召回数	簇大小标准差	容量违规率/%	日内移动耗时/min	时段违规数	审查重试次数
完整系统	0.1	8.2	0.5	3	98	0.1	0.3
A1	1.9	3.4	0.5	3	98	0.1	0.3
A2	0.1	8.2	2.9	41	93	0.1	1.5
A3	0.1	8.2	0.5	3	91	1.5	1.7
A4	0.1	8.2	0.5	3	154	0.1	0.3

由表 5-10 可见，关闭双层 RAG 后（A1），POI 幻觉数由 0.1 升至 1.9，非知识库城市的有效景点召回数由 8.2 降至 3.4。失去真实攻略片段作为参照，种子景点提取节点只能凭模型自身的参数化记忆生成景点名，而模型对长尾目的地与近期变化的覆盖本就薄弱，幻觉因此明显增多。与此同时，节点也无法向高德 POI 接口提供有效的搜索关键词，未收录城市的召回链路近乎中断。A2 至 A4 三项消融分别对应行程编排中的聚类、分桶与路径优化三个环节。以标准 K-Means 替代均衡 K-Means（A2）后，簇大小标准差由 0.5 升至 2.9，容量违规率由 3% 升至 41%。同一城市的热门景点经纬度往往高度集中，标准 K-Means 仅以簇内方差最小化为目标，很容易将其划入同一大簇，超过四成的簇因此超出每日景点数上限，行程审查节点也被反复触发。取消时段分桶（A3）后，时段违规数由 0.1 升至 1.5，审查重试次数由 0.3 升至 1.7，缺少时段桶约束，求解器仅有移动代价一个优化目标，容易将夜间景点排到日间，违规一旦超过重试上限便会带入最终输出。值得注意的是，A2 与 A3 的日内移动耗时反而略低于完整系统，这反映了完整系统在容量与时段约束下牺牲了部分移动效率以换取语义合理性，符合预期权衡。取消路径优化（A4）后，日内景点间移动耗时由 98 分钟升至 154 分钟，其余指标几乎不变；约 57% 的增量直接反映了 TSP 与 2-opt 的作用，也说明路径优化职能单纯、对其他模块几乎不产生副作用。

综合以上结果表明，系统各模块职责边界清晰、相互解耦良好，不存在隐性的强耦合依赖，验证了第三章与第四章所述分层架构设计的合理性；同时也从模块粒度证实了各组件的独立贡献。消融实验结果与 5.2.4 节的对比测试结论相互补充，共同印证了所提系统在功能完整性与结果正确性方面的有效性。

5.6 本章小结

本章通过系统的实验设计，对提出的旅行规划智能体架构进行了全面的有效性验证。首先明确了测试环境、配置与方法，确保对比实验的公平性。其次分别进行了系统功能

测试、对比测试以及消融实验。综合 5.3 至 5.5 节的测试结果，本章三组实验在不同粒度上对系统进行了完整评估。在端到端功能层面，四组用例分别覆盖了知识库典型规划、非知识库城市兜底、约束缺失协商与非规划意图沉淀四类 workflow 分支，所有分支均按设计输出预期结果，端到端时延稳定在约 80 秒，外部接口在所测用例上未出现导致整体阻断的失败。在横向对比层面，本系统在 POI 有效率、POI 幻觉数、时段违规数、交通住宿信息真实性等关键质量指标上相对于纯大语言模型基线呈现数量级差距，预算可核算且不超支比例也有明显提升，时延上的劣势主要源自多阶段编排与多次外部接口调用，与任务定位相符。在模块贡献层面，消融实验进一步显示双层 RAG、均衡 K-Means、时段分桶与桶内 TSP 与跨日 2-opt 路径优化各自承担了独立、可识别的作用，模块间未出现强耦合依赖。三组实验共同支持了本章关于架构有效性与模块分工合理性的结论。

第六章 总结与展望

6.1 设计成果与主要结论

针对自由行规划中信息分散、多约束求解困难，以及通用大语言模型在空间推理、实时数据获取、领域知识与跨会话记忆方面存在的不足，本文设计并实现了一套以 LangGraph 与 MCP 为基础的智能旅行规划智能体系统。经过需求分析、架构设计、模块实现与实验测试，研究取得的主要成果如下。

(1) 实现了一个端到端、模块化的旅行规划智能体原型。系统以 React/Vite 前端为交互入口，以 FastAPI 为后端框架，核心是基于 LangGraph 构建的、包含五个主阶段的有状态工作流。系统接收用户的自然语言输入后，依次完成意图识别、约束抽取与协商、并行数据获取、行程编排、餐饮住宿匹配与预算自检，最终输出包含逐日景点路线、真实交通班次与票价、多档住宿、餐饮建议及预算明细的完整方案。MCP 工具层、RAG 知识层、行程算法层与记忆系统等模块均以类型化共享状态（TravelGraphState）为载体完成数据交换与协同，表明此类复杂智能体系统在工程上可以通过清晰的模块划分加以实现。

(2) 基于 MCP 协议完成了对异构外部服务的稳定集成，提升了信息的真实性与实时性。针对地图、票务、攻略等多源数据接入问题，系统以 MCP 协议作为统一的工具接入层，设计了可同时适配 SSE 与 STDIO 两种传输方式的抽象客户端基类，使工具的定义、发现与调用逻辑同宿主应用解耦。在客户端层进一步叠加请求重试、熔断保护与字段裁剪归一化，以应对外部接口的超时、限流与格式差异。测试用例中 MCP 工具调用成功率为 100%，单一接口的偶发故障未引发整体服务中断。借助高德地图接口获取的真实行程时间矩阵与 POI 详情，以及 12306、途牛接口提供的实时票价与库存，系统生成的交通、住宿信息真实性由纯大语言模型基线的 16% 提升至 96%，缓解了大模型在实时数据获取上的固有限制。

(3) 设计并验证了双层 RAG 与语义协同的行程编排方案，抑制了幻觉并保障了行程的空间合理性。为弥补大模型在垂直领域知识与空间组合推理上的不足，本文提出两项设计。一方面是双层检索增强生成架构，将高确定性的结构化城市知识库与覆盖长尾内容的非结构化攻略向量库相结合；在检索环节，先由模型生成与目标语料形态相近的假设性文本作为查询以提升召回，再以重排序模型对候选片段进行精排，随后由结构

化抽取模块将检索结果整理为经验提示与风险提示供下游使用。消融实验显示，关闭该机制后 POI 幻觉数由 0.1 个/方案上升至 1.9 个/方案；对比测试中系统的 POI 有效率高于纯大语言模型基线。另一方面是将大模型的语义理解与传统运筹优化相结合的行程编排算法，把多日规划分解为景点分日聚类与日内路径优化两个子问题：前者采用基于真实行程时间矩阵的带容量约束均衡 K-Means，使各日景点数均衡且地理集中；后者先将景点按上午、午后弹性与夜间三类时段分桶，再在桶内求解旅行商问题，从而在结构上避免时段安排违规。消融实验表明，移除该算法会使日均容量违规率升至 41%、时段违规升至 1.5 个/方案、日内移动耗时增加 57%，说明将空间组合推理交由专用算法处理是必要且有效的。

(4) 实现了长短期记忆分离的偏好管理，支持跨会话的个性化服务。为突破大模型对话的无状态限制，系统将短期会话状态与长期用户偏好分别存储：前者由 LangGraph 的检查点机制维护，后者以用户为单位持久化为独立的 JSON 文件，两者物理隔离，避免相互污染。偏好抽取任务被合并入意图识别节点，在一次大模型推理中同时完成意图判断与偏好提取，不引入额外调用。已记录的偏好（如倾向的酒店品牌、交通方式）在新会话开始时以系统消息形式注入上下文，使下游节点据此给出个性化推荐。端到端测试验证了从偏好陈述、持久化记录到跨会话复用的完整链路，为低开销的个性化偏好管理提供了一种可行实现。

(5) 通过分层实验验证了整体架构的有效性与各模块的独立贡献。本文采用三次实验体系提供量化依据。端到端功能测试覆盖典型规划、长尾城市召回、约束协商与偏好更新四类场景，验证了工作流的正确性，平均响应时延约 80 秒，配合流式渲染后具备实际可用性。对比测试在 12 条用例的评测集上，将本系统与仅依赖单次提示的大语言模型基线逐项对照，结果显示系统在 POI 有效率、幻觉数、交通住宿信息真实性、时段合规性与预算可控性等指标上均优于基线，其中部分指标差距达到数量级。消融实验则以单变量替换的方式，分别量化了双层 RAG、均衡 K-Means 容量约束、时段分桶与路径优化算法的作用；数据表明任一模块缺失都会导致相应指标退化，从模块层面说明了各项设计的必要性。

综合来看，本文通过 LangGraph 状态图进行流程编排与状态管理，借助 MCP 协议实现异构外部工具的解耦与稳定集成，以大模型负责语义理解、传统算法负责空间优化的分工方式协同求解，并以双层 RAG 与分层记忆系统分别补充领域知识与用户偏好，从而在多个环节弥补了通用大语言模型在复杂垂直任务中的能力短板，得到结果较为可

靠、交互较为连贯且具备一定实用价值的系统。上述工作可为智能行程规划方向的研究，以及同类复杂决策支持系统的构建提供参考。

6.2 设计特色与创新

本设计的特色围绕旅行规划场景中的若干核心问题展开，主要体现在以下四个方面。

(1) 基于状态图的工作流编排。不同于以链式调用或简单循环为主的智能体框架，本设计借助 LangGraph 将一个允许条件回环的有向状态图应用于旅行规划，把整个过程划分为意图识别、并行获取、行程编排、行程审查与餐饮住宿匹配五个职责明确的阶段。其中，分支处理由条件路由完成，数据获取阶段通过并行展开缩短等待时延，失败节点则借助局部重试回环实现就近恢复与故障隔离。相比隐式的过程控制，这种显式且可视化的流程建模提升了长链路任务的可控性、可观测性与可维护性。

(2) 面向异构工具集成的协议化与容错设计。考虑到外部服务接口差异较大且运行并不稳定，本设计不采用将模型与工具紧耦合的方案，而是以与模型无关的 MCP 协议作为统一的抽象层。在传输层面，设计了同时适配 SSE 与 STDIO 两种方式的客户端抽象；在协议层之上，进一步集成字段裁剪、请求重试与熔断降级三类机制。由此，工具层在保障信息实时性与真实性的同时，也具备了应对生产环境波动的能力，为智能体与真实数据源之间的稳定交互提供了一种可行的工程实现。

(3) 分层协同的求解架构。本设计将旅行规划拆分为四个相互协作的层次：由大语言模型承担意图、约束与偏好抽取的语义理解层，由双层 RAG 负责注入领域事实与经验的知识增强层，由传统算法完成聚类与路径求解的空间优化层，以及由 MCP 工具获取实时数据的服务执行层。这一安排让大语言模型专注于其擅长的自然语言理解与开放式生成，而把坐标计算、组合优化等确定性较强的任务交由专用算法处理，使语义理解能力与精确计算能力各司其职、相互补充。

(4) 面向旅游领域的知识增强与偏好管理。在知识增强方面，针对旅游领域结构化必去清单与非结构化攻略文本并存的特点，本设计采用双层检索并辅以桥接机制：先由结构化知识库召回核心景点作为种子，再以这些种子驱动向量库进行语义扩展，从而在保证核心景点确定性召回的同时，覆盖长尾信息。在记忆管理方面，本设计将短期会话状态与长期用户偏好分别存储，并把偏好抽取合并入意图识别节点，在不增加额外模型调用的前提下持续记录用户偏好；新会话开始时，已记录的偏好以系统消息形式注入上下文，使下游节点无需用户重复说明即可给出个性化推荐，从而增强了交互的连续性。

6.3 不足与展望

本研究实现了预定目标，并通过实验验证了所提架构的有效性。但受研究周期与资源条件所限，工作仍存在若干不足，这些不足也对应着后续可以继续推进的方向。

（1）数据源的覆盖范围与动态性有待拓展。当前系统接入的数据源以国内服务为主，尚未覆盖跨境机票、国际酒店与签证政策等面向境外出行的信息；对景区实时客流、临时闭馆、突发天气等分钟级变化的感知与应对机制也未纳入。后续可借助 MCP 协议的可插拔特性接入国际化数据源，并在工作流中引入基于事件触发的行程调整机制，使系统不再局限于一次性的静态方案生成，而能够对出行过程中的动态变化作出响应。

（2）交互方式与使用体验有待丰富。目前的交互以文本为主，虽然采用 SSE 流式输出在一定程度上缓解了等待感，但仍难以适应移动、车载等场景下的交互需求。后续可引入多模态大语言模型，支持用户通过图片、语音乃至短视频表达偏好或查询信息，并以语音交互降低操作成本，从而覆盖更多样的出行场景。

（3）评测体系与用户研究有待完善。本文的评测集虽然涵盖了多种场景，但规模仍然有限，也尚未在真实用户群体中开展较大规模的可用性测试与 A/B 测试。后续可参考已有的旅行规划评测基准（如 TravelPlanner、TripTailor）构建更系统的评测体系，并补充以用户满意度、方案可执行性与偏好命中率为核心的人工评估，为系统的迭代优化提供更可靠的依据。

（4）系统性能与部署形态有待提升。当前系统为单机原型，长期记忆基于文件存储，难以支撑高并发访问，平均约 80 秒的响应时延也存在压缩空间。后续可将系统改造为微服务架构，把记忆系统迁移至数据库，对 MCP 服务端进行独立部署与扩展，并在关键节点前引入异步队列与缓存，以提高吞吐量、降低时延，使系统更接近实际应用环境的要求。

参考文献

- [1] Xie J, Zhang K, Chen J, et al. TravelPlanner: A Benchmark for Real-World Planning with Language Agents [C]. In: Proceedings of the 41st International Conference on Machine Learning. PMLR, 2024: 54590-54613.
- [2] Weng L. LLM Powered Autonomous Agents [EB/OL]. (2023-06-23)[2026-05-21]. <https://lilianweng.github.io/posts/2023-06-23-agent/>.
- [3] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents [J]. Frontiers of Computer Science, 2024, 18(6): 186345.
- [4] Xi Z, Chen W, Guo X, et al. The rise and potential of large language model based agents: A survey [J]. Science China Information Sciences, 2025, 68(2): 121101.
- [5] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [J]. Advances in neural information processing systems, 2022, 35: 24824-24837.
- [6] Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models [J]. arXiv preprint arXiv:2210.03629, 2022.
- [7] Shinn N, Cassano F, Gopinath A, 等. Reflexion: Language agents with verbal reinforcement learning [J]. Advances in neural information processing systems, 2023, 36: 8634-8652.
- [8] Wu Q, Bansal G, Zhang J, et al. Autogen: Enabling next-gen LLM applications via multi-agent conversations [C]. In: First conference on language modeling. 2024.
- [9] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta programming for a multi-agent collaborative framework [C]. In: International Conference on Learning Representations: Vol. 2024. 2024: 23247-23275.
- [10] Chase H. LangChain [CP/OL]. 2022 [2026-05-21]. <https://github.com/langchain-ai/langchain>.
- [11] crewAIInc/crewAI [CP/OL]. crewAI, 2026. <https://github.com/crewAIInc/crewAI>.
- [12] microsoft/semantic-kernel [CP/OL]. Microsoft, 2026 [2026-05-21]. <https://github.com/microsoft/semantic-kernel>.
- [13] langchain-ai/langgraph [CP/OL]. LangChain, 2026 [2026-03-02]. <https://github.com/langchain-ai/langgraph>.
- [14] Wang L, Xu W, Lan Y, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models [C]. In: Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers). 2023: 2609-2634.
- [15] Shen Y, Song K, Tan X, et al. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face [J]. Advances in Neural Information Processing Systems, 2023, 36: 38154-38180.
- [16] Liu B, Jiang Y, Zhang X, et al. Llm+ p: Empowering large language models with optimal planning proficiency [J]. arXiv preprint arXiv:2304.11477, 2023.
- [17] 秦龙, 武万森, 刘丹, 等. 基于大语言模型的复杂任务自主规划处理框架 [J]. 自动化学报, 2024, 50 (4): 862-872.
- [18] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [C]. In: Advances in Neural Information Processing Systems: Vol. 33. Curran Associates, Inc., 2020: 9459-9474.
- [19] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey [EB]. arXiv, 2024.
- [20] Asai A, Wu Z, Wang Y, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection [J]. International Conference on Learning Representations, 2024, 2024: 9112-9141.

-
- [21] Edge D, Trinh H, Cheng N, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization [EB]. arXiv, 2025.
- [22] Xiao S, Liu Z, Zhang P, et al. C-Pack: Packed Resources For General Chinese Embeddings [C]. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York, NY, USA: Association for Computing Machinery, 2024: 641-649.
- [23] Schick T, Dwivedi-Yu J, Dessì R, 等. Toolformer: Language models can teach themselves to use tools [J]. Advances in neural information processing systems, 2023, 36: 68539-68551.
- [24] Qin Y, Liang S, Ye Y, 等. Toolllm: Facilitating large language models to master 16000+ real-world apis [C]. 见: International Conference on Learning Representations: 卷 2024. 2024: 9695-9717.
- [25] Patil S G, Zhang T, Wang X, et al. Gorilla: Large language model connected with massive apis [J]. Advances in Neural Information Processing Systems, 2024, 37: 126544-126565.
- [26] Function calling | OpenAI API [EB/OL]. [2026-05-20]. <https://developers.openai.com/api/docs/guides/function-calling>.
- [27] Introducing the Model Context Protocol [EB/OL]. [2026-05-20]. <https://www.anthropic.com/news/model-context-protocol>.
- [28] Koren Y, Bell R, Volinsky C. Matrix factorization techniques for recommender systems [J]. Computer, 2009, 42(8): 30-37.
- [29] Cheng C, Yang H, King I, et al. Fused matrix factorization with geographical and social influence in location-based social networks [C]. In: Proceedings of the AAAI conference on artificial intelligence: Vol. 26. 2012: 17-23.
- [30] Yang D, Zhang D, Zheng V W, et al. Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs [J]. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2014, 45(1): 129-142.
- [31] Sun K, Qian T, Chen T, et al. Where to go next: Modeling long-and short-term user preferences for point-of-interest recommendation [C]. In: Proceedings of the AAAI conference on artificial intelligence: Vol. 34. 2020: 214-221.
- [32] 贾中浩, 宾辰忠, 古天龙, 等. 基于知识图谱和用户长短期偏好的个性化景点推荐 [J]. 智能系统学报, 2020, 15 (5): 990-997.
- [33] 刘志中, 李林霞, 孟令强. 基于混合图神经网络的个性化 POI 推荐方法研究 [J]. 南京大学学报 (自然科学), 2023, 59 (3): 373-387.
- [34] Tang Y, Wang Z, Qu A, et al. ItiNera: Integrating Spatial Optimization with Large Language Models for Open-domain Urban Itinerary Planning [C]. In: Dernoncourt F, Preoțiuc-Pietro D, Shimorina A, eds. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. Miami, Florida, US: Association for Computational Linguistics, 2024: 1413-1432.
- [35] Optimizing LLM-based trip planning [EB/OL]. [2026-05-20]. <https://research.google/blog/optimizing-llm-based-trip-planning/>.
- [36] Wang K, Shen Y, Lv C, et al. TripTailor: A Real-World Benchmark for Personalized Travel Planning [C]. In: Che W, Nabende J, Shutova E, et al., eds. Findings of the Association for Computational Linguistics: ACL 2025. Vienna, Austria: Association for Computational Linguistics, 2025: 9705-9723.
